

File: consumo_lib.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi

```
import sys
import cv2
import time
import numpy as np
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QLabel, QGroupBox, QGridLayout, QLineEdit,
                             QComboBox, QListWidget, QListWidgetItem, QFileDialog, QMessageBox, QTabWidget,
                             QSplitter, QFrame, QTableWidget, QTableWidgetItem, QHeaderView, QDialog)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer
from PyQt6.QtGui import QPixmap, QImage, QFont, QAction

from aoi_lib import CNCAOIController, InspectionPosition
from grbl_streamer import GrblStreamer

import logging
logger = logging.getLogger("consumo_lib")
logger.setLevel(logging.DEBUG)
# Se necessário, adicione um handler:
if not logger.handlers:
    ch = logging.StreamHandler()
    ch.setLevel(logging.DEBUG)
    formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
    ch.setFormatter(formatter)
    logger.addHandler(ch)

class ImageViewerWidget(QWidget):
    """Widget para exibir imagens capturadas pela câmera"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)

        # Label para exibir a imagem
        self.image_label = QLabel()
        self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.image_label.setText("Nenhuma imagem capturada")
        self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
        self.image_label.setMinimumSize(400, 300)

        # Informações da imagem
        self.info_label = QLabel("Informações da imagem:")

        self.layout.addWidget(self.image_label)
        self.layout.addWidget(self.info_label)

    def display_image(self, image, info_text=None):
        """Mostra uma imagem no widget"""
        if image is None:
            self.image_label.setText("Imagem inválida")
```

```
return
```

```
# Converte imagem OpenCV para QPixmap
h, w, c = image.shape
bytes_per_line = 3 * w
q_img = QImage(image.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
pixmap = QPixmap.fromImage(q_img)
```

```
# Redimensiona se for muito grande
if pixmap.width() > 800 or pixmap.height() > 600:
    pixmap = pixmap.scaled(800, 600, Qt.AspectRatioMode.KeepAspectRatio)
```

```
self.image_label.setPixmap(pixmap)
```

```
# Atualiza informações
if info_text:
    self.info_label.setText(info_text)
else:
    self.info_label.setText(f"Imagem: {w}x{h}px")
```

```
class PositionListWidget(QWidget):
```

```
    """Widget para gerenciar lista de posições"""
```

```
    position_selected = pyqtSignal(object) # Emite a posição selecionada
```

```
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)
```

```
    # Título
    title_label = QLabel("Posições de Inspeção")
    title_font = QFont()
    title_font.setBold(True)
    title_label.setFont(title_font)
```

```
    # Lista de posições
    self.positions_list = QListWidget()
    self.positions_list.currentItemChanged.connect(self.on_position_selected)
```

```
    # Botões
    buttons_layout = QHBoxLayout()
    self.add_position_btn = QPushButton("Adicionar Posição Atual")
    self.remove_position_btn = QPushButton("Remover")
    buttons_layout.addWidget(self.add_position_btn)
    buttons_layout.addWidget(self.remove_position_btn)
```

```
    self.layout.addWidget(title_label)
    self.layout.addWidget(self.positions_list)
    self.layout.addLayout(buttons_layout)
```

```
    # Dicionário para mapear itens da lista para objetos de posição
    self.positions = {}
```

```

def add_position(self, position):
    """Adiciona uma posição à lista"""
    item_text = f"{{position.name}} ({{position.x:.2f}}, {{position.y:.2f}})"
    item = QListWidgetItem(item_text)
    self.positions_list.addItem(item)
    self.positions[id(item)] = position

def remove_selected_position(self):
    """Remove a posição selecionada"""
    current_item = self.positions_list.currentItem()
    if current_item:
        position = self.positions.pop(id(current_item))
        row = self.positions_list.row(current_item)
        self.positions_list.takeItem(row)
        return position
    return None

def clear_positions(self):
    """Limpa todas as posições"""
    self.positions_list.clear()
    self.positions = {}

def get_all_positions(self):
    """Retorna todas as posições"""
    return list(self.positions.values())

def on_position_selected(self, current, previous):
    """Manipula evento de seleção de posição"""
    if current and current in self.positions:
        self.position_selected.emit(self.positions[current])

class SequenceControlWidget(QWidget):
    """Widget para controlar a execução da sequência"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)

        # Grupo de sequência
        sequence_group = QGroupBox("Controle de Sequência")
        sequence_layout = QGridLayout()

        # Nome da sequência
        sequence_layout.addWidget(QLabel("Nome:"), 0, 0)
        self.sequence_name = QLineEdit("Sequência PCB")
        sequence_layout.addWidget(self.sequence_name, 0, 1)

        # Botões de controle
        self.create_sequence_btn = QPushButton("Criar Sequência")
        self.run_sequence_btn = QPushButton("Executar Sequência")
        self.stop_sequence_btn = QPushButton("Parar")

```

```

self.stop_sequence_btn.setEnabled(False)

sequence_layout.addWidget(self.create_sequence_btn, 1, 0)
sequence_layout.addWidget(self.run_sequence_btn, 1, 1)
sequence_layout.addWidget(self.stop_sequence_btn, 2, 0, 1, 2)

# Status
sequence_layout.addWidget(QLabel("Status:"), 3, 0)
self.sequence_status = QLabel("Pronto")
sequence_layout.addWidget(self.sequence_status, 3, 1)

sequence_group.setLayout(sequence_layout)

# Grupo de arquivo
file_group = QGroupBox("Salvar/Carregar")
file_layout = QVBoxLayout()
self.save_btn = QPushButton("Salvar Programa (JSON)")
self.load_btn = QPushButton("Carregar Programa (JSON)")
self.save_gcode_btn = QPushButton("Exportar para G-CODE")
self.load_gcode_btn = QPushButton("Importar de G-CODE")
file_layout.addWidget(self.save_btn)
file_layout.addWidget(self.load_btn)
file_layout.addWidget(self.save_gcode_btn)
file_layout.addWidget(self.load_gcode_btn)

file_group.setLayout(file_layout)

self.layout.addWidget(sequence_group)
self.layout.addWidget(file_group)
self.layout.addStretch()

```

```

class PositionRegistryWidget(QWidget):
    """Widget for showing registered positions"""
    def __init__(self, controller, parent=None):
        super().__init__(parent)
        self.controller = controller
        self.positions = [] # List of registered positions
        self.setup_ui()

    def setup_ui(self):
        layout = QVBoxLayout(self)

        # Title
        title_label = QLabel("Registered Positions")
        title_font = QFont()
        title_font.setBold(True)
        title_label.setFont(title_font)
        layout.addWidget(title_label)

        # Registered positions table
        self.positions_table = QTableWidgetItem(0, 3)

```

```
self.positions_table.setHorizontalHeaderLabels(["Name", "X (mm)", "Y (mm)"])
self.positions_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
layout.addWidget(self.positions_table)
```

```
# Buttons
```

```
buttons_layout = QHBoxLayout()
```

```
self.delete_button = QPushButton("Delete Selected")
self.delete_button.clicked.connect(self.delete_position)
```

```
self.create_sequence_btn = QPushButton("Create Sequence")
```

```
buttons_layout.addWidget(self.delete_button)
buttons_layout.addWidget(self.create_sequence_btn)
```

```
layout.addLayout(buttons_layout)
```

```
def add_position(self, name, x, y, image=None):
```

```
    """Add a position to the registry"""
```

```
    # Create position object
```

```
    position = {
        'name': name,
        'x': x,
        'y': y,
        'image': image,
    }
```

```
    # Add to internal list
```

```
    self.positions.append(position)
```

```
    # Add to table
```

```
    row = self.positions_table.rowCount()
    self.positions_table.insertRow(row)
    self.positions_table.setItem(row, 0, QTableWidgetItem(name))
    self.positions_table.setItem(row, 1, QTableWidgetItem(f"{x:.3f}")
    self.positions_table.setItem(row, 2, QTableWidgetItem(f"{y:.3f}"))
```

```
def delete_position(self):
```

```
    """Delete selected position"""
```

```
    selected_rows = self.positions_table.selectedItems()
```

```
    if not selected_rows:
```

```
        return
```

```
    row = selected_rows[0].row()
```

```
    if row >= 0 and row < len(self.positions):
```

```
        del self.positions[row]
```

```
        self.positions_table.removeRow(row)
```

```
def clear_positions(self):
```

```
    """Clear all positions"""
```

```
    self.positions.clear()
```

```
while self.positions_table.rowCount() > 0:
    self.positions_table.removeRow(0)
```

```
class CameraPreviewWidget(QWidget):
```

```
    """Widget for displaying camera preview and capturing images"""
```

```
    image_captured = pyqtSignal(object, str) # Emits the captured image and position name
```

```
    def __init__(self, controller, parent=None):
```

```
        super().__init__(parent)
```

```
        self.controller = controller
```

```
        self.current_image = None
```

```
        self.preview_timer = QTimer(self)
```

```
        self.preview_timer.timeout.connect(self.update_preview)
```

```
        self.setup_ui()
```

```
    def setup_ui(self):
```

```
        layout = QVBoxLayout(self)
```

```
        # Preview area - Ajustar para usar mais espaço
```

```
        self.image_label = QLabel()
```

```
        self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
        self.image_label.setText("Camera Preview")
```

```
        self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
```

```
        self.image_label.setMinimumSize(600, 450) # Aumentar tamanho mínimo
```

```
        # Camera controls
```

```
        controls_layout = QHBoxLayout()
```

```
        self.start_preview_btn = QPushButton("Start Preview")
```

```
        self.start_preview_btn.clicked.connect(self.start_preview)
```

```
        self.stop_preview_btn = QPushButton("Stop Preview")
```

```
        self.stop_preview_btn.clicked.connect(self.stop_preview)
```

```
        self.stop_preview_btn.setEnabled(False)
```

```
        controls_layout.addWidget(self.start_preview_btn)
```

```
        controls_layout.addWidget(self.stop_preview_btn)
```

```
        # Position capture layout
```

```
        capture_layout = QHBoxLayout()
```

```
        self.position_name = QLineEdit()
```

```
        self.position_name.setPlaceholderText("Position Name")
```

```
        self.capture_button = QPushButton("Capture Image & Register Position")
```

```
        self.capture_button.clicked.connect(self.capture_image)
```

```
        capture_layout.addWidget(self.position_name)
```

```
        capture_layout.addWidget(self.capture_button)
```

```
        # Adicionar elementos ao layout principal
```

```
        layout.addWidget(self.image_label, 1) # Proporção 1 para permitir expansão
```

```
        layout.addLayout(controls_layout)
```

```
        layout.addLayout(capture_layout)
```

```
    def start_preview(self):
```

```
        """Start camera preview"""
```

```

if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
    QMessageBox.warning(self, "Error", "Camera not connected")
    return
self.preview_timer.start(100) # Update every 100ms
self.start_preview_btn.setEnabled(False)
self.stop_preview_btn.setEnabled(True)

def stop_preview(self):
    """Stop camera preview"""
    self.preview_timer.stop()
    self.start_preview_btn.setEnabled(True)
    self.stop_preview_btn.setEnabled(False)

def update_preview(self):
    """Update the camera preview"""
    try:
        image = self.controller.camera.capture()
        if image is not None:
            self.display_image(image)
            self.current_image = image
    except Exception as e:
        print(f"Error updating preview: {e}")
        self.stop_preview()

def capture_image(self):
    """Capture an image and emit signal with position name"""
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Error", "Camera not connected")
        return
    position_name = self.position_name.text()
    if not position_name:
        QMessageBox.warning(self, "Error", "Please enter a position name")
        return
    try:
        image = self.controller.camera.capture()
        if image is not None:
            self.display_image(image)
            self.current_image = image
            self.image_captured.emit(image, position_name)
            self.position_name.clear()
        else:
            QMessageBox.warning(self, "Error", "Failed to capture image")
    except Exception as e:
        QMessageBox.warning(self, "Error", f"Capture error: {e}")

def display_image(self, image):
    """Display an image in the preview area with a crosshair in the center"""
    if image is None:
        return

    # Criar uma cópia da imagem para não modificar a original

```

```
display_img = image.copy()
```

```
# Desenhar a cruz vermelha no centro
```

```
h, w, _ = display_img.shape
```

```
center_x, center_y = w // 2, h // 2
```

```
# Parâmetros da cruz
```

```
color = (0, 0, 255) # Vermelho em BGR
```

```
thickness = 2
```

```
length = min(w, h) // 20 # 5% do tamanho da dimensão menor
```

```
# Desenhar a cruz
```

```
# Linha horizontal
```

```
cv2.line(display_img,
```

```
        (center_x - length, center_y),
```

```
        (center_x + length, center_y),
```

```
        color, thickness)
```

```
# Linha vertical
```

```
cv2.line(display_img,
```

```
        (center_x, center_y - length),
```

```
        (center_x, center_y + length),
```

```
        color, thickness)
```

```
# Converter a imagem OpenCV para QPixmap
```

```
h, w, c = display_img.shape
```

```
bytes_per_line = 3 * w
```

```
q_img = QImage(display_img.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
```

```
pixmap = QPixmap.fromImage(q_img)
```

```
# Obter tamanho do widget de exibição
```

```
label_width = self.image_label.width()
```

```
label_height = self.image_label.height()
```

```
# Redimensionar a imagem para caber no espaço disponível mantendo proporções
```

```
pixmap = pixmap.scaled(label_width, label_height,
```

```
                        Qt.AspectRatioMode.KeepAspectRatio,
```

```
                        Qt.TransformationMode.SmoothTransformation)
```

```
self.image_label.setPixmap(pixmap)
```

```
def resizeEvent(self, event):
```

```
    """Override do evento de redimensionamento para ajustar a imagem quando o widget for redimensionado"""
```

```
    super().resizeEvent(event)
```

```
    if self.current_image is not None:
```

```
        self.display_image(self.current_image)
```

```
class MovementControlWidget(QWidget):
```

```
    """Widget for controlling CNC movement (jog)"""
```

```
    def __init__(self, controller, parent=None):
```

```
        super().__init__(parent)
```

```
        self.controller = controller
```

```
self.setup_ui()
```

```
def setup_ui(self):
```

```
    layout = QVBoxLayout(self)
```

```
    # Group box for movement controls
```

```
    movement_group = QGroupBox("Movement Controls")
```

```
    movement_layout = QGridLayout()
```

```
    # Directional control buttons
```

```
    self.up_button = QPushButton("↑")
```

```
    self.down_button = QPushButton("↓")
```

```
    self.left_button = QPushButton("←")
```

```
    self.right_button = QPushButton("→")
```

```
    # Style the buttons
```

```
    for btn in [self.up_button, self.down_button, self.left_button, self.right_button]:
```

```
        btn.setMinimumSize(50, 50)
```

```
        font = QFont()
```

```
        font.setBold(True)
```

```
        font.setPointSize(16)
```

```
        btn.setFont(font)
```

```
    # Connect press/release events for continuous movement
```

```
    self.up_button.pressed.connect(lambda: self.start_movement("Y", 1))
```

```
    self.up_button.released.connect(self.stop_movement)
```

```
    self.down_button.pressed.connect(lambda: self.start_movement("Y", -1))
```

```
    self.down_button.released.connect(self.stop_movement)
```

```
    self.left_button.pressed.connect(lambda: self.start_movement("X", -1))
```

```
    self.left_button.released.connect(self.stop_movement)
```

```
    self.right_button.pressed.connect(lambda: self.start_movement("X", 1))
```

```
    self.right_button.released.connect(self.stop_movement)
```

```
    # Add buttons to grid
```

```
    movement_layout.addWidget(self.up_button, 0, 1)
```

```
    movement_layout.addWidget(self.left_button, 1, 0)
```

```
    movement_layout.addWidget(self.right_button, 1, 2)
```

```
    movement_layout.addWidget(self.down_button, 2, 1)
```

```
    # Step size and feed rate controls
```

```
    step_layout = QHBoxLayout()
```

```
    step_layout.addWidget(QLabel("Step Size:"))
```

```
    self.step_size = QLineEdit("1.0")
```

```
    step_layout.addWidget(self.step_size)
```

```
    step_layout.addWidget(QLabel("mm"))
```

```
    feed_layout = QHBoxLayout()
```

```
    feed_layout.addWidget(QLabel("Feed Rate:"))
```

```
    self.feed_rate = QLineEdit("1000")
```

```
    feed_layout.addWidget(self.feed_rate)
```

```
    feed_layout.addWidget(QLabel("mm/min"))
```

```

# Add step and feed rate controls
movement_layout.addLayout(step_layout, 3, 0, 1, 3)
movement_layout.addLayout(feed_layout, 4, 0, 1, 3)

# Movement mode (G90/G91)
mode_layout = QHBoxLayout()
self.mode_absolute = QPushButton("Passo a Passo (G90)")
self.mode_absolute.setCheckable(True)
self.mode_absolute.clicked.connect(lambda: self.set_motion_mode("G90"))

self.mode_relative = QPushButton("Contínuo (G91)")
self.mode_relative.setCheckable(True)
self.mode_relative.setChecked(True) # Default to relative mode
self.mode_relative.clicked.connect(lambda: self.set_motion_mode("G91"))

mode_layout.addWidget(self.mode_absolute)
mode_layout.addWidget(self.mode_relative)
movement_layout.addLayout(mode_layout, 5, 0, 1, 3)

movement_group.setLayout(movement_layout)
layout.addWidget(movement_group)

def start_movement(self, axis, direction):
    """Inicia movimento no eixo e direção especificados"""
    logger.debug(f"MOVIMENTO: Iniciando movimento: eixo={axis}, direção={direction}")

    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
        logger.warning("MOVIMENTO: CNC não conectado")
        QMessageBox.warning(self, "Error", "CNC not connected")
        return

    try:
        # Captura posição antes do movimento para verificação posterior
        original_position = self.controller.cnc.get_current_position()
        logger.info(f"MOVIMENTO: Posição antes do movimento: {original_position}")

        step_size = float(self.step_size.text())
        feed_rate = float(self.feed_rate.text())

        # Verifica o modo atual
        is_absolute_mode = self.mode_absolute.isChecked()
        logger.debug(f"MOVIMENTO: Modo atual: {'Absoluto (G90)' if is_absolute_mode else 'Relativo (G91)'}")

        if is_absolute_mode:
            # CORREÇÃO: Implementação melhorada para modo absoluto (passo a passo)
            # Enviamos uma sequência atômica de comandos para evitar problemas de sincronização

            # 1. Calculamos o tamanho do passo com a direção
            distance = step_size * direction

```

2. Definimos a sequência completa de comandos

```
commands = [  
    "G91", # Modo relativo  
    f"G1 {axis.upper()}{distance} F{feed_rate}", # Movimento com G1  
    "G90" # Volta ao modo absoluto  
]
```

3. Enviamos os comandos com intervalos controlados

logger.debug(f"MOVIMENTO: Enviando sequência de comandos para movimento absoluto")

for i, cmd in enumerate(commands):

 QTimer.singleShot(50 * i, lambda c=cmd: self.controller.cnc.grbl.send_immediately(c))

 logger.debug(f"MOVIMENTO: Programado envio de comando: {cmd}")

else:

 # Modo contínuo - usa comando \$J= para jog

 distance = 100 * direction # Distância grande para movimento contínuo

 if axis.upper() == 'X':

 command = f"\$J=G91 X{distance} F{feed_rate}"

 else: # Y

 command = f"\$J=G91 Y{distance} F{feed_rate}"

 logger.debug(f"MOVIMENTO: Enviando comando contínuo: {command}")

 self.controller.cnc.grbl.send_immediately(command)

Agenda verificação após intervalo adequado para permitir que o movimento ocorra

Usar intervalo maior (500ms) para dar tempo suficiente ao movimento físico

QTimer.singleShot(500, lambda: self.verify_position_changed(axis, direction, original_position))

except Exception as e:

 logger.error(f"MOVIMENTO: Erro ao iniciar movimento: {str(e)}")

 QMessageBox.warning(self, "Error", f"Error starting movement: {str(e)}")

def verify_position_changed(self, axis, direction, original_position):

 """Verifica se a posição realmente mudou após comando de movimento"""

 try:

 current_position = self.controller.cnc.get_current_position()

 expected_axis = 'x' if axis.upper() == 'X' else 'y'

 logger.info(f"VERIFICAÇÃO: Movimento {axis}{'+' if direction > 0 else '-'}: "

 f"Original={original_position[expected_axis]}, "

 f"Atual={current_position[expected_axis]}")

 if original_position[expected_axis] == current_position[expected_axis]:

 logger.warning(f"VERIFICAÇÃO: Posição {expected_axis} NÃO MUDOU após comando!")

 # Forçar atualização de status para tentar obter a posição atual

 if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:

 logger.debug(f"VERIFICAÇÃO: Enviando comando de status ? para atualizar posição")

 self.controller.cnc.grbl.send_immediately("?")

except Exception as e:

```
logger.error(f"VERIFICAÇÃO: Erro ao verificar posição: {e}")
```

```
def stop_movement(self):
```

```
    """Para o movimento"""
```

```
    logger.debug("MOVIMENTO: Parando movimento")
```

```
    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
```

```
        return
```

```
    try:
```

```
        # Verifica o modo atual
```

```
        is_absolute_mode = self.mode_absolute.isChecked()
```

```
    if is_absolute_mode:
```

```
        # No modo absoluto (passo a passo), não precisamos parar o movimento
```

```
        # porque ele já deve ter sido concluído quando o botão é liberado
```

```
        logger.debug("MOVIMENTO: Modo absoluto, ignorando comando de parada")
```

```
        return
```

```
    else:
```

```
        # Apenas no modo contínuo enviamos os comandos de parada
```

```
        # Envia comando de feed hold para parar movimento
```

```
        logger.debug("MOVIMENTO: Enviando comando de parada (!)")
```

```
        self.controller.cnc.grbl.send_immediately("!")
```

```
        # Após uma breve pausa, envia comando resume para liberar o estado hold
```

```
        logger.debug("MOVIMENTO: Agendando comando de retomada (~) após pausa")
```

```
        QTimer.singleShot(100, lambda: self._execute_resume_and_update())
```

```
    except Exception as e:
```

```
        logger.error(f"MOVIMENTO: Erro ao parar movimento: {e}")
```

```
def _execute_resume_and_update(self):
```

```
    """Executa o resumo após parada e força atualização de posição em sequência"""
```

```
    try:
```

```
        # Primeiro envia o comando para retomar após hold
```

```
        self.controller.cnc.grbl.send_immediately("~")
```

```
        logger.debug("MOVIMENTO: Enviado comando de retomada (~)")
```

```
        # Agenda uma série de comandos de status para garantir obtenção da posição atualizada
```

```
        # Primeira verificação após 150ms
```

```
        QTimer.singleShot(150, lambda: self._force_position_update(1))
```

```
    except Exception as e:
```

```
        logger.error(f"MOVIMENTO: Erro ao executar sequência de retomada: {e}")
```

```
def _force_position_update(self, attempt=1):
```

```
    """Força múltiplas atualizações de posição para garantir precisão
```

```
    Args:
```

```
        attempt: Número da tentativa atual (para limitar tentativas)
```

```
    """
```

```
    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
```

```
return
```

```
try:
```

```
    # Envia comando de status para obter posição atualizada
    logger.debug(f"MOVIMENTO: Forçando atualização de posição - tentativa {attempt}")
    self.controller.cnc.grbl.send_immediately("?")
```

```
    # Se ainda estamos dentro do limite de tentativas, agenda outra verificação
```

```
    if attempt < 3:
```

```
        # Aumento progressivo do tempo entre tentativas
```

```
        delay = 150 + (attempt * 50)
```

```
        QTimer.singleShot(delay, lambda: self._force_position_update(attempt + 1))
```

```
except Exception as e:
```

```
    logger.error(f"MOVIMENTO: Erro ao forçar atualização de posição: {e}")
```

```
def _force_position_update(self, attempt=1):
```

```
    """Força múltiplas atualizações de posição para garantir precisão
```

```
    Args:
```

```
        attempt: Número da tentativa atual (para limitar tentativas)
```

```
    """
```

```
    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
```

```
        return
```

```
    try:
```

```
        # Envia comando de status para obter posição atualizada
```

```
        logger.debug(f"MOVIMENTO: Forçando atualização de posição - tentativa {attempt}")
```

```
        self.controller.cnc.grbl.send_immediately("?")
```

```
        # Se ainda estamos dentro do limite de tentativas, agenda outra verificação
```

```
        if attempt < 3:
```

```
            # Aumento progressivo do tempo entre tentativas (150ms, 200ms, 250ms)
```

```
            delay = 150 + (attempt * 50)
```

```
            QTimer.singleShot(delay, lambda: self._force_position_update(attempt + 1))
```

```
except Exception as e:
```

```
    logger.error(f"MOVIMENTO: Erro ao forçar atualização de posição: {e}")
```

```
def set_motion_mode(self, mode):
```

```
    """Set the motion mode (G90/G91)"""
```

```
    if not self.controller.cnc.is_connected:
```

```
        return
```

```
    if mode == "G90":
```

```
        self.mode_absolute.setChecked(True)
```

```
        self.mode_relative.setChecked(False)
```

```
    else:
```

```
        self.mode_absolute.setChecked(False)
```

```
        self.mode_relative.setChecked(True)
```

```
try:
    self.controller.cnc.send_command(mode)
except Exception as e:
    print(f"Error setting motion mode: {e}")
```

```
class AOIControllerApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Controle de Inspeção Óptica Automatizada")
        self.setGeometry(100, 100, 1200, 800)

        # Inicializa o controlador AOI
        self.controller = CNCAOIController()
        self.current_sequence = None
        self.is_running_sequence = False

        # Variável para armazenar o último valor de posição (para comparação)
        self.last_logged_position = None

        # Configuração da interface
        self.setup_ui()
        # Configuração do menu
        self.setup_menu()
        # Timer para atualizar a posição – agora conectamos a um método que loga a ação
        self.update_timer = QTimer(self)
        self.update_timer.timeout.connect(self.on_update_timer)
        self.update_timer.start(500) # Atualiza a cada 500ms
        # Variável para armazenar o último valor de posição (para comparação)
        self.last_logged_position = None

    def on_update_timer(self):
        logger.debug("Timer fired: atualizando tela de posição da head.")
        self.update_position_display()

    def setup_ui(self):
        # Widget central
        central_widget = QWidget()
        self.setCentralWidget(central_widget)

        # Layout principal
        main_layout = QVBoxLayout(central_widget)

        # Grupo de conexão
        connection_group = QGroupBox("Conexão")
        connection_layout = QGridLayout()

        # CNC Connection
        connection_layout.addWidget(QLabel("Porta CNC:"), 0, 0)
        self.cnc_port_combo = QComboBox()
        self.refresh_ports()
        connection_layout.addWidget(self.cnc_port_combo, 0, 1)
```

```

self.connect_cnc_btn = QPushButton("Conectar CNC")
self.connect_cnc_btn.clicked.connect(self.connect_cnc)
connection_layout.addWidget(self.connect_cnc_btn, 0, 2)

# Camera connection
connection_layout.addWidget(QLabel("Câmera ID:"), 1, 0)
self.camera_id_combo = QComboBox()
self.camera_id_combo.addItems(["0", "1", "2", "3"])
connection_layout.addWidget(self.camera_id_combo, 1, 1)

self.connect_camera_btn = QPushButton("Conectar Câmera")
self.connect_camera_btn.clicked.connect(self.connect_camera)
connection_layout.addWidget(self.connect_camera_btn, 1, 2)

# Refresh ports button
self.refresh_ports_btn = QPushButton("Atualizar Portas")
self.refresh_ports_btn.clicked.connect(self.refresh_ports)
connection_layout.addWidget(self.refresh_ports_btn, 0, 3)

# Test camera button
self.test_camera_btn = QPushButton("Testar Câmera")
self.test_camera_btn.clicked.connect(self.test_camera)
connection_layout.addWidget(self.test_camera_btn, 1, 3)

connection_group.setLayout(connection_layout)
main_layout.addWidget(connection_group)

# Grupo de Calibração de Movimento
calibration_group = QGroupBox("Calibração de Movimento")
calibration_layout = QHBoxLayout()

calibration_layout.addWidget(QLabel("Pulsos/Revolução:"))
self.pulses_input = QLineEdit("400")
calibration_layout.addWidget(self.pulses_input)

calibration_layout.addWidget(QLabel("Passo do Fuso (mm):"))
self.fuso_input = QLineEdit("5")
calibration_layout.addWidget(self.fuso_input)

self.apply_calibration_btn = QPushButton("Aplicar Calibração")
self.apply_calibration_btn.clicked.connect(self.apply_calibration)
calibration_layout.addWidget(self.apply_calibration_btn)

calibration_group.setLayout(calibration_layout)
main_layout.addWidget(calibration_group)

# Splitter para dividir a interface em painéis
splitter = QSplitter(Qt.Orientation.Horizontal)

# Painel esquerdo: controles e lista de posições

```

```

left_panel = QWidget()
left_layout = QVBoxLayout(left_panel)

# Informações de posição
position_group = QGroupBox("Posição Atual")
position_layout = QGridLayout()

# Exibe os valores X, Y e status
position_layout.addWidget(QLabel("X:"), 0, 0)
self.x_position = QLabel("0.000 mm")
position_layout.addWidget(self.x_position, 0, 1)

position_layout.addWidget(QLabel("Y:"), 1, 0)
self.y_position = QLabel("0.000 mm")
position_layout.addWidget(self.y_position, 1, 1)

position_layout.addWidget(QLabel("Status:"), 2, 0)
self.cnc_status = QLabel("Desconectado")
position_layout.addWidget(self.cnc_status, 2, 1)

# NOVO: Botão para setar a posição atual como zero
self.set_zero_btn = QPushButton("Setar Posição Zero")
self.set_zero_btn.clicked.connect(self.set_zero_position)
position_layout.addWidget(self.set_zero_btn, 3, 0, 1, 2)

position_group.setLayout(position_layout)
left_layout.addWidget(position_group)

# Widget de lista de posições
self.position_list_widget = PositionListWidget()
self.position_list_widget.add_position_btn.clicked.connect(self.add_current_position)
self.position_list_widget.remove_position_btn.clicked.connect(self.remove_position)
self.position_list_widget.position_selected.connect(self.on_position_selected)

left_layout.addWidget(self.position_list_widget)

# Controle de sequência
self.sequence_widget = SequenceControlWidget()
self.sequence_widget.create_sequence_btn.clicked.connect(self.create_sequence)
self.sequence_widget.run_sequence_btn.clicked.connect(self.run_sequence)
self.sequence_widget.stop_sequence_btn.clicked.connect(self.stop_sequence)
self.sequence_widget.save_btn.clicked.connect(self.save_program)
self.sequence_widget.load_btn.clicked.connect(self.load_program)
self.sequence_widget.save_gcode_btn.clicked.connect(self.save_gcode)
self.sequence_widget.load_gcode_btn.clicked.connect(self.load_gcode)

left_layout.addWidget(self.sequence_widget)

# Painel direito: visualizador de imagem e resultados
right_panel = QTabWidget()

```

```

# Aba de visualização de imagem
self.image_viewer = ImageViewerWidget()
right_panel.addTab(self.image_viewer, "Visualizador de Imagem")

# Aba de resultados
results_widget = QWidget()
results_layout = QVBoxLayout(results_widget)

self.results_table = QTableWidgetItem(0, 3)
self.results_table.setHorizontalHeaderLabels(["Posição", "Timestamp", "Status"])
self.results_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)

results_layout.addWidget(QLabel("Resultados da Inspeção:"))
results_layout.addWidget(self.results_table)

right_panel.addTab(results_widget, "Resultados")

# MOVER PARA AQUI: Adicionar a aba de Câmera & Movimento (após definir right_panel)
# Tab para Camera & Movement
camera_movement_tab = QWidget()
camera_movement_layout = QHBoxLayout(camera_movement_tab)

# Left side: controls and position registry
cm_left_panel = QWidget()
cm_left_layout = QVBoxLayout(cm_left_panel)

# Movement controls
self.movement_widget = MovementControlWidget(self.controller)
cm_left_layout.addWidget(self.movement_widget)

# Position registry
self.position_registry = PositionRegistryWidget(self.controller)
self.position_registry.create_sequence_btn.clicked.connect(self.create_sequence_from_registry)
cm_left_layout.addWidget(self.position_registry)

# Right side: camera preview
self.camera_preview = CameraPreviewWidget(self.controller)
self.camera_preview.image_captured.connect(self.on_image_captured)

# Define proporções para os painéis - dar mais espaço para a visualização da câmera
camera_movement_layout.addWidget(cm_left_panel, 1) # Proporção 1
camera_movement_layout.addWidget(self.camera_preview, 3) # Proporção 3 (mais espaço)

# Agora é seguro adicionar a nova aba ao right_panel que já foi definido
right_panel.addTab(camera_movement_tab, "Câmera & Movimento")

# Adiciona painéis ao splitter
splitter.addWidget(left_panel)
splitter.addWidget(right_panel)
splitter.setSizes([400, 800])

```

```

main_layout.addWidget(splitter)

# Barra de status
self.statusBar().showMessage("Pronto para conectar")

def setup_menu(self):
    """Configura o menu da aplicação"""
    menubar = self.menuBar()

    # Menu de Arquivo
    file_menu = menubar.addMenu('&Arquivo')

    exit_action = QAction('Sair', self)
    exit_action.setShortcut('Ctrl+Q')
    exit_action.triggered.connect(self.close)
    file_menu.addAction(exit_action)

    # Menu de Ferramentas
    tools_menu = menubar.addMenu('&Ferramentas')

    calibration_action = QAction('Calibração CNC', self)
    calibration_action.triggered.connect(self.show_calibration_dialog)
    tools_menu.addAction(calibration_action)

    # Menu de Ajuda
    help_menu = menubar.addMenu('&Ajuda')

    about_action = QAction('Sobre', self)
    about_action.triggered.connect(self.show_about_dialog)
    help_menu.addAction(about_action)

def show_calibration_dialog(self):
    """Mostra um diálogo para configuração de calibração"""
    dialog = QDialog(self)
    dialog.setWindowTitle("Calibração do Sistema CNC")
    dialog.setMinimumWidth(500)

    layout = QVBoxLayout(dialog)

    # Grupo de parâmetros físicos
    param_group = QGroupBox("Parâmetros da Máquina")
    param_layout = QGridLayout(param_group)

    param_layout.addWidget(QLabel("Pulsos por Revolução:"), 0, 0)
    pulses_input = QLineEdit(self.pulses_input.text())
    param_layout.addWidget(pulses_input, 0, 1)

    param_layout.addWidget(QLabel("Passo do Fuso (mm):"), 1, 0)
    fuso_input = QLineEdit(self.fuso_input.text())
    param_layout.addWidget(fuso_input, 1, 1)

```

```
param_layout.addWidget(QLabel("Steps/mm calculado:"), 2, 0)
steps_mm_result = QLabel("Calculando...")
param_layout.addWidget(steps_mm_result, 2, 1)
```

```
# Atualiza o cálculo quando os valores mudam
```

```
def update_calculation():
```

```
    try:
```

```
        pulses = float(pulses_input.text())
```

```
        fuso = float(fuso_input.text())
```

```
        steps_mm = pulses / fuso
```

```
        steps_mm_result.setText(f"{steps_mm:.3f} steps/mm")
```

```
    except:
```

```
        steps_mm_result.setText("Erro no cálculo")
```

```
pulses_input.textChanged.connect(update_calculation)
```

```
fuso_input.textChanged.connect(update_calculation)
```

```
update_calculation() # Executa o cálculo inicial
```

```
layout.addWidget(param_group)
```

```
# Botões de ação
```

```
buttons_layout = QHBoxLayout()
```

```
apply_btn = QPushButton("Aplicar Parâmetros")
```

```
def apply_and_close():
```

```
    self.pulses_input.setText(pulses_input.text())
```

```
    self.fuso_input.setText(fuso_input.text())
```

```
    dialog.accept()
```

```
    self.apply_calibration()
```

```
apply_btn.clicked.connect(apply_and_close)
```

```
test_btn = QPushButton("Testar Calibração")
```

```
test_btn.clicked.connect(lambda: [dialog.accept(), self.show_calibration_test_dialog()])
```

```
cancel_btn = QPushButton("Cancelar")
```

```
cancel_btn.clicked.connect(dialog.reject)
```

```
buttons_layout.addWidget(apply_btn)
```

```
buttons_layout.addWidget(test_btn)
```

```
buttons_layout.addWidget(cancel_btn)
```

```
layout.addLayout(buttons_layout)
```

```
dialog.exec()
```

```
def show_about_dialog(self):
```

```
    """Mostra informações sobre o aplicativo"""
```

```
    QMessageBox.about(self, "Sobre HesaiVision",
```

```
        "HesaiVision v1.0\n\n"
```

```
        "Sistema de Inspeção Óptica Automatizada\n"
```

```
        "Desenvolvido para controle de CNC com GRBL\n\n"
```

```
def save_gcode(self):
    """Salva a sequência atual como arquivo G-CODE"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return

    from aoi_lib.gcode_manager import GCodeManager
    gcode_manager = GCodeManager()

    filename, _ = QFileDialog.getSaveFileName(
        self, "Salvar como G-CODE", "", "Arquivos G-CODE (*.gcode *.nc *.ngc)"
    )

    if filename:
        if not filename.lower().endswith(('gcode', '.nc', '.ngc')):
            filename += '.gcode'

        if gcode_manager.save_gcode_to_file(self.current_sequence, filename):
            self.statusBar().showMessage(f"G-CODE salvo em {filename}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao salvar o arquivo G-CODE")

def apply_calibration(self):
    """
    Aplica a calibração de movimento com base nos valores informados pelo usuário.
    Envia comandos diretos para o GRBL para configurar os steps/mm.
    """
    if not self.controller.cnc.is_connected or not self.controller.cnc.grbl:
        QMessageBox.warning(self, "Erro", "CNC não conectada. Conecte primeiro.")
        return

    try:
        pulses = float(self.pulses_input.text())
        fuso_pass = float(self.fuso_input.text())

        # Calcula steps/mm: (pulsos por revolução) / (passo do fuso em mm)
        steps_per_mm = pulses / fuso_pass

        # Armazena o valor calculado
        self.controller.cnc.steps_to_mm_factor = fuso_pass / pulses

        # Envia comandos para configurar o GRBL
        logger.info(f"CALIBRAÇÃO: Configurando steps/mm para {steps_per_mm}")

        # Verifica se o usuário deseja realmente enviar estes valores
        reply = QMessageBox.question(
            self,
            "Confirmar Calibração",
            f"Deseja enviar os seguintes parâmetros para o GRBL?\n\n"
        )
```

```

f"Steps/mm eixo X: {steps_per_mm:.3f}\n"
f"Steps/mm eixo Y: {steps_per_mm:.3f}\n\n"
f"Issso irá alterar a configuração do controlador.",
QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
)

if reply == QMessageBox.StandardButton.Yes:
    # Comandos para configurar os eixos X e Y
    self.controller.cnc.grbl.send_immediately(f"$100={steps_per_mm:.3f}")
    QTimer.singleShot(100, lambda: self.controller.cnc.grbl.send_immediately(f"$101={steps_per_mm:.3f}"))

    # Solicita ao usuário que faça um teste de calibração
    QTimer.singleShot(500, self.show_calibration_test_dialog)

    self.statusBar().showMessage(f"Calibração aplicada: {steps_per_mm:.3f} steps/mm")
else:
    self.statusBar().showMessage("Calibração cancelada pelo usuário")

except ValueError:
    QMessageBox.warning(self, "Erro", "Valores de calibração inválidos.")

def show_calibration_test_dialog(self):
    """
    Exibe um diálogo para testar a calibração atual.
    """
    dialog = QDialog(self)
    dialog.setWindowTitle("Teste de Calibração")
    dialog.setMinimumWidth(400)

    layout = QVBoxLayout(dialog)

    # Instruções
    instructions = QLabel(
        "Para testar a calibração:\n\n"
        "1. Coloque um papel milimetrado ou uma régua sob a cabeça da máquina\n"
        "2. Escolha uma distância de teste\n"
        "3. Clique em 'Mover X' ou 'Mover Y' para testar cada eixo\n"
        "4. Verifique se o deslocamento físico corresponde ao valor escolhido\n"
        "5. Se necessário, ajuste os valores de calibração e aplique novamente"
    )
    instructions.setWordWrap(True)
    layout.addWidget(instructions)

    # Distância de teste
    test_layout = QHBoxLayout()
    test_layout.addWidget(QLabel("Distância de teste:"))
    distance_input = QLineEdit("10")
    test_layout.addWidget(distance_input)
    test_layout.addWidget(QLabel("mm"))
    layout.addLayout(test_layout)

```

```
# Botões de teste
```

```
buttons_layout = QHBoxLayout()
```

```
move_x_btn = QPushButton("Mover X")
```

```
move_x_btn.clicked.connect(lambda: self.test_calibration_move(0, float(distance_input.text())))
```

```
move_y_btn = QPushButton("Mover Y")
```

```
move_y_btn.clicked.connect(lambda: self.test_calibration_move(1, float(distance_input.text())))
```

```
reset_position_btn = QPushButton("Zerar Posição")
```

```
reset_position_btn.clicked.connect(self.set_zero_position)
```

```
buttons_layout.addWidget(move_x_btn)
```

```
buttons_layout.addWidget(move_y_btn)
```

```
buttons_layout.addWidget(reset_position_btn)
```

```
layout.addLayout(buttons_layout)
```

```
# Resultados
```

```
result_group = QGroupBox("Resultados")
```

```
result_layout = QVBoxLayout(result_group)
```

```
self.calibration_test_result = QLabel("Execute um teste para ver os resultados")
```

```
result_layout.addWidget(self.calibration_test_result)
```

```
layout.addWidget(result_group)
```

```
# Botões de controle
```

```
control_layout = QHBoxLayout()
```

```
close_btn = QPushButton("Concluir")
```

```
close_btn.clicked.connect(dialog.accept)
```

```
control_layout.addWidget(close_btn)
```

```
layout.addLayout(control_layout)
```

```
dialog.exec()
```

```
def test_calibration_move(self, axis, distance):
```

```
    """
```

```
    Realiza um movimento de teste para calibração.
```

```
    Args:
```

```
        axis: 0 para X, 1 para Y
```

```
        distance: Distância em mm para mover
```

```
    """
```

```
    if not self.controller.cnc.is_connected:
```

```
        QMessageBox.warning(self, "Erro", "CNC não conectada")
```

```
        return
```

```
    try:
```

```
        # Captura posição inicial
```

```
initial_position = self.controller.cnc.get_current_position()
```

```
# Prepara o comando
```

```
axis_name = "X" if axis == 0 else "Y"
```

```
# Define o modo absoluto para garantir precisão
```

```
self.controller.cnc.grbl.send_immediately("G90")
```

```
# Calcula a posição absoluta a ser atingida
```

```
target_position = {}
```

```
target_position['x'] = initial_position['x'] + distance if axis == 0 else initial_position['x']
```

```
target_position['y'] = initial_position['y'] + distance if axis == 1 else initial_position['y']
```

```
# Envia o movimento como coordenada absoluta
```

```
cmd = f"G1 {axis_name}{target_position[axis_name.lower()]} F500"
```

```
logger.info(f"CALIBRAÇÃO: Enviando comando de teste: {cmd}")
```

```
self.controller.cnc.grbl.send_immediately(cmd)
```

```
# Aguarda um pouco para o movimento ser concluído
```

```
QTimer.singleShot(1500, lambda: self.verify_calibration_result(axis, initial_position, distance))
```

```
except Exception as e:
```

```
logger.error(f"CALIBRAÇÃO: Erro no teste de calibração: {e}")
```

```
QMessageBox.warning(self, "Erro", f"Erro no teste: {str(e)}")
```

```
def verify_calibration_result(self, axis, initial_position, expected_distance):
```

```
    """
```

```
    Verifica o resultado do teste de calibração.
```

```
    Args:
```

```
        axis: 0 para X, 1 para Y
```

```
        initial_position: Posição antes do movimento
```

```
        expected_distance: Distância esperada do movimento
```

```
    """
```

```
    try:
```

```
        # Força atualização da posição
```

```
        self.controller.cnc.grbl.send_immediately("?")
```

```
        # Aguarda um pouco para receber a atualização
```

```
        time.sleep(0.2)
```

```
        # Obtem posição atual
```

```
        current_position = self.controller.cnc.get_current_position()
```

```
        # Calcula o deslocamento real
```

```
        axis_name = "x" if axis == 0 else "y"
```

```
        actual_distance = current_position[axis_name] - initial_position[axis_name]
```

```
        # Calcula o erro
```

```
        error = actual_distance - expected_distance
```

```
        error_percent = (error / expected_distance) * 100
```

```

# Atualiza o resultado do teste
result_text = (
    f"Eixo: {axis_name.upper()}\n"
    f"Movimento comandado: {expected_distance:.3f} mm\n"
    f"Movimento real: {actual_distance:.3f} mm\n"
    f"Erro: {error:.3f} mm ({error_percent:.2f}%)\n\n"
)

if abs(error_percent) < 1:
    result_text += "■ Calibração excelente (erro < 1%)"
elif abs(error_percent) < 5:
    result_text += "✓ Calibração aceitável (erro < 5%)"
else:
    result_text += "■ Calibração insatisfatória - Ajuste os parâmetros"

if hasattr(self, 'calibration_test_result'):
    self.calibration_test_result.setText(result_text)

logger.info(f"CALIBRAÇÃO: Resultado do teste - {result_text}")

except Exception as e:
    logger.error(f"CALIBRAÇÃO: Erro ao verificar resultado: {e}")

def set_zero_position(self):
    """Define a posição atual como zero."""
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "CNC não conectada")
        return
    if self.controller.cnc.set_zero():
        self.statusBar().showMessage("Posição zero setada.")
        self.update_position_display()
    else:
        QMessageBox.warning(self, "Erro", "Falha ao setar posição zero.")

def on_image_captured(self, image, position_name):
    """Handle captured image and register position"""
    # Display in image viewer
    self.image_viewer.display_image(image, f"Image: {position_name}")

    # Get current position
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Error", "CNC not connected")
        return

    current_pos = self.controller.cnc.get_current_position()

    # Register position with image
    self.position_registry.add_position(
        position_name,
        current_pos['x'],

```

```
        current_pos['y'],
        image
    )
```

```
self.statusBar().showMessage(f'Position '{position_name}' registered at X:{current_pos['x']:.3f}, Y:{current_pos['y']:.3f}')
```

```
def create_sequence_from_registry(self):
```

```
    """Create a sequence from registered positions"""
```

```
    if not self.position_registry.positions:
```

```
        QMessageBox.warning(self, "Warning", "No positions registered")
```

```
    return
```

```
    # Get sequence name
```

```
    sequence_name = self.sequence_widget.sequence_name.text()
```

```
    if not sequence_name:
```

```
        QMessageBox.warning(self, "Warning", "Please enter a sequence name")
```

```
    return
```

```
    # Clear existing positions in the position list widget
```

```
    self.position_list_widget.clear_positions()
```

```
    # Create positions for the sequence
```

```
    positions = []
```

```
    for pos in self.position_registry.positions:
```

```
        # Create position object with camera parameters
```

```
        camera_params = {'has_image': pos['image'] is not None}
```

```
        inspection_pos = InspectionPosition(
```

```
            pos['name'],
```

```
            pos['x'],
```

```
            pos['y'],
```

```
            camera_params
```

```
        )
```

```
        positions.append(inspection_pos)
```

```
    # Also add to the position list widget
```

```
    self.position_list_widget.add_position(inspection_pos)
```

```
    # Create the sequence
```

```
    self.current_sequence = self.controller.create_sequence(sequence_name, positions)
```

```
    # Update UI
```

```
    self.sequence_widget.sequence_status.setText(f'Created: {len(positions)} positions')
```

```
    self.statusBar().showMessage(f'Sequence '{sequence_name}' created with {len(positions)} positions')
```

```
    # Ask if user wants to save as G-CODE
```

```
    reply = QMessageBox.question(
```

```
        self,
```

```
        "Save G-CODE",
```

```
        "Do you want to save this sequence as G-CODE?",
```

```
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
```

)

```
if reply == QMessageBox.StandardButton.Yes:  
    self.save_gcode()
```

```
def load_gcode(self):
```

```
    """Carrega uma sequência a partir de um arquivo G-CODE"""
```

```
    from aoi_lib.gcode_manager import GCodeManager
```

```
    gcode_manager = GCodeManager()
```

```
    filename, _ = QFileDialog.getOpenFileName(  
        self, "Abrir G-CODE", "", "Arquivos G-CODE (*.gcode *.nc *.ngc)"
```

```
)
```

```
    if filename:
```

```
        # Carrega o G-CODE
```

```
        result = gcode_manager.read_gcode_file(filename)
```

```
    if result:
```

```
        # Limpa posições atuais
```

```
        self.position_list_widget.clear_positions()
```

```
        # Cria sequência a partir dos dados do G-CODE
```

```
        positions = []
```

```
        for pos_data in result['positions']:
```

```
            # Cria objeto de posição
```

```
            position = InspectionPosition(  
                pos_data['name'],
```

```
                pos_data['x'],
```

```
                pos_data['y'],
```

```
                pos_data['camera_params']
```

```
            )
```

```
        # Adiciona à lista visual
```

```
        self.position_list_widget.add_position(position)
```

```
        # Adiciona à lista interna
```

```
        positions.append(position)
```

```
        # Cria a sequência
```

```
        self.current_sequence = self.controller.create_sequence(  
            result['sequence_name'], positions
```

```
        )
```

```
        # Atualiza interface
```

```
        self.sequence_widget.sequence_name.setText(result['sequence_name'])
```

```
        self.sequence_widget.sequence_status.setText(  
            f"Carregada do G-CODE: {len(positions)} posições"
```

```
        )
```

```
        self.statusBar().showMessage(f"G-CODE carregado de {filename}")
```

else:

QMessageBox.critical(self, "Erro", "Falha ao carregar o arquivo G-CODE")

def refresh_ports(self):

"""Atualiza a lista de portas seriais disponíveis"""

import serial.tools.list_ports

self.cnc_port_combo.clear()

ports = [port.device for port in serial.tools.list_ports.comports()]

if ports:

self.cnc_port_combo.addItems(ports)

Verificar se COM9 está na lista e selecionar

com9_index = self.cnc_port_combo.findText("COM9")

if com9_index >= 0:

self.cnc_port_combo.setCurrentIndex(com9_index)

self.statusBar().showMessage("Porta COM9 detectada")

else:

self.statusBar().showMessage("Nenhuma porta serial encontrada")

def get_current_position(self):

"""

Retorna a posição atual do CNC.

Certifica-se de retornar uma cópia do dicionário para evitar alteração acidental.

"""

logger.debug(f"get_current_position chamado, retornando: {self.current_position}")

return self.current_position.copy() if hasattr(self, 'current_position') else {'x': 0, 'y': 0, 'z': 0}

def connect_cnc(self):

"""Conecta à máquina CNC usando a biblioteca grbl-streamer"""

if hasattr(self.controller.cnc, 'is_connected') and self.controller.cnc.is_connected:

Desconectar

try:

if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:

self.controller.cnc.grbl.poll_stop()

self.controller.cnc.grbl.disconnect()

self.controller.cnc.grbl = None

self.controller.cnc.is_connected = False

self.connect_cnc_btn.setText("Conectar CNC")

self.cnc_status.setText("Desconectado")

self.statusBar().showMessage("CNC desconectada")

except Exception as e:

self.statusBar().showMessage(f"Erro ao desconectar: {str(e)}")

else:

Conectar

port = self.cnc_port_combo.currentText()

if not port:

QMessageBox.warning(self, "Erro", "Selecione uma porta serial")

return

```
self.statusBar().showMessage(f"Conectando à CNC na porta {port}...")
```

```
try:
```

```
    # Define função de callback para eventos do GrblStreamer
```

```
    def grbl_callback(eventstring, *data):
```

```
        logger.debug(f"CALLBACK: Evento '{eventstring}' recebido com data: {data}")
```

```
    if eventstring == "on_stateupdate":
```

```
        if len(data) >= 3:
```

```
            state = data[0]
```

```
            mpos = data[1] # Tupla de coordenadas da máquina (x, y, z)
```

```
            wpos = data[2] # Tupla de coordenadas de trabalho (x, y, z)
```

```
            # Log detalhado para depuração
```

```
            logger.debug(f"CALLBACK DETALHADO: state={state}, mpos={mpos}, wpos={wpos}")
```

```
            # Atualiza estado da máquina
```

```
            old_state = self.controller.cnc.machine_status if hasattr(self.controller.cnc, 'machine_status') else None
```

```
            self.controller.cnc.machine_status = state
```

```
            if old_state != state:
```

```
                logger.debug(f"CALLBACK: Estado da máquina mudou de '{old_state}' para '{state}'")
```

```
            # CORREÇÃO CRÍTICA: Verifica tipo e estrutura dos dados antes de usar
```

```
            if isinstance(wpos, (list, tuple)) and len(wpos) >= 2:
```

```
                try:
```

```
                    # CORREÇÃO: Verifica se posição recebida está em formato de tupla ou lista
```

```
                    old_position = None
```

```
                    if hasattr(self.controller.cnc, 'current_position'):
```

```
                        old_position = self.controller.cnc.current_position.copy()
```

```
                    # Copia informação para evitar referência compartilhada
```

```
                    new_position = {
```

```
                        'x': float(wpos[0]),
```

```
                        'y': float(wpos[1]),
```

```
                        'z': float(wpos[2]) if len(wpos) > 2 else 0
```

```
                    }
```

```
                    # Registra TODA mudança de posição, mesmo pequena
```

```
                    if old_position != new_position:
```

```
                        logger.info(f"POSIÇÃO ALTERADA: {old_position} → {new_position}")
```

```
                    # Atualiza a posição no controlador apenas se mudou
```

```
                    self.controller.cnc.current_position = new_position
```

```
            except (ValueError, TypeError, IndexError) as e:
```

```
                logger.error(f"CALLBACK: Erro ao processar posição: {e}, wpos={wpos}")
```

```
            elif isinstance(wpos, dict):
```

```
                # Se já for um dicionário, o que pode ocorrer em algumas versões
```

```
                logger.debug(f"CALLBACK: wpos já é um dicionário")
```

```
                self.controller.cnc.current_position = wpos.copy()
```

else:

```
logger.error(f"CALLBACK: Formato inválido para wpos: {type(wpos)}, valor: {wpos}")
```

```
# Adiciona mais log para outros eventos importantes
```

```
elif eventstring == "on_write":
```

```
logger.debug(f"CALLBACK: Comando enviado: {data[0] if data else 'vazio'}")
```

```
elif eventstring == "on_read":
```

```
# Depuração de respostas do GRBL, especialmente status <...>
```

```
response = data[0] if data else "vazio"
```

```
if response.startswith("<") and "MPos" in response:
```

```
logger.debug(f"CALLBACK: Resposta status raw do GRBL: {response}")
```

```
# CORREÇÃO: Inicializa o GrblStreamer com callback, SEM passar 'port' ao construtor
```

```
self.controller.cnc.grbl = GrblStreamer(grbl_callback)
```

```
# Configura logging (opcional)
```

```
self.controller.cnc.grbl.setup_logging()
```

```
# Conecta à porta usando o método correto cnect() (não connect)
```

```
logger.debug(f"CONEXÃO: Conectando à porta {port}")
```

```
self.controller.cnc.grbl.cnect(port, 115200)
```

```
# Desbloqueia a máquina
```

```
logger.debug("CONEXÃO: Enviando comando de desbloqueio $X")
```

```
self.controller.cnc.grbl.send_immediately("$X")
```

```
# Inicia em modo relativo para jog
```

```
logger.debug("CONEXÃO: Configurando modo relativo G91")
```

```
self.controller.cnc.grbl.send_immediately("G91")
```

```
# Inicia verificação de status
```

```
logger.debug("CONEXÃO: Iniciando polling de status")
```

```
self.controller.cnc.grbl.poll_start()
```

```
# Atualiza o estado da conexão
```

```
self.controller.cnc.is_connected = True
```

```
self.controller.cnc.machine_status = "Idle" # Estado inicial presumido
```

```
# Atualiza UI
```

```
self.connect_cnc_btn.setText("Desconectar CNC")
```

```
self.cnc_status.setText("Conectado")
```

```
self.statusBar().showMessage(f"CNC conectada na porta {port}")
```

```
except Exception as e:
```

```
logger.error(f"CONEXÃO: Falha ao conectar: {str(e)}")
```

```
QMessageBox.critical(self, "Erro", f"Falha ao conectar a CNC: {str(e)}")
```

```
def connect_camera(self):
```

```
    """Conecta à câmera"""
```

```
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:
```

```
        # Desconectar
```

```

self.controller.camera.disconnect()
self.connect_camera_btn.setText("Conectar Câmera")
self.statusBar().showMessage("Câmera desconectada")
else:
    # Conectar
    try:
        camera_id = int(self.camera_id_combo.currentText())

        self.statusBar().showMessage(f"Conectando à câmera ID {camera_id}...")

        if self.controller.connect_camera(camera_id):
            self.connect_camera_btn.setText("Desconectar Câmera")
            self.statusBar().showMessage(f"Câmera ID {camera_id} conectada")
        else:
            QMessageBox.critical(self, "Erro", f"Falha ao conectar à câmera: {self.controller.camera.last_error}")
    except ValueError:
        QMessageBox.warning(self, "Erro", "ID de câmera inválido")

def test_camera(self):
    """Testa a captura de imagem da câmera"""
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Aviso", "Câmera não conectada")
    return

    image = self.controller.camera.capture()

    if image is not None:
        self.image_viewer.display_image(image, "Imagem de teste da câmera")
        self.statusBar().showMessage("Imagem capturada com sucesso")
    else:
        QMessageBox.warning(self, "Erro", f"Falha ao capturar imagem: {self.controller.camera.last_error}")

def update_position_display(self):
    """Atualiza a exibição da posição atual"""
    if not self.controller.cnc.is_connected:
        logger.debug("update_position_display: CNC não conectada.")
        return
    try:
        position = self.controller.cnc.get_current_position()
        logger.debug("update_position_display: posição obtida do CNC: %s", position)
    except Exception as e:
        logger.error("update_position_display: erro ao obter posição: %s", e)
        return
    # Verificação para detectar se a posição não mudou (mesmo após movimento)
    if self.last_logged_position is not None:
        if position == self.last_logged_position:
            logger.warning("update_position_display: posição inalterada: %s", position)
        else:
            logger.debug("update_position_display: posição mudou de %s para %s",
                          self.last_logged_position, position)
    else:

```

```

logger.debug("update_position_display: nenhuma posição anterior registrada.")

self.last_logged_position = position.copy()

# Atualiza os labels da interface
self.x_position.setText(f'{{position['x']:.3f}} mm")
self.y_position.setText(f'{{position['y']:.3f}} mm")
self.cnc_status.setText(self.controller.cnc.machine_status)

def add_current_position(self):
    """Adiciona a posição atual à lista"""
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "CNC não conectada")
        return

    position_name, ok = QInputDialog.getText(self, "Nova Posição", "Nome da posição:")

    if ok and position_name:
        position = self.controller.add_current_position(position_name)
        self.position_list_widget.add_position(position)
        self.statusBar().showMessage(f'Posição '{position_name}' adicionada")

def remove_position(self):
    """Remove a posição selecionada"""
    position = self.position_list_widget.remove_selected_position()

    if position:
        self.controller.position_manager.remove_position(position.name)
        self.statusBar().showMessage(f'Posição '{position.name}' removida")

def on_position_selected(self, position):
    """Manipula a seleção de uma posição"""
    # Poderia mover para esta posição, mostrar detalhes, etc.
    self.statusBar().showMessage(f'Posição selecionada: {position.name} ({{position.x:.3f}}, {{position.y:.3f}})")

def create_sequence(self):
    """Cria uma nova sequência com as posições atuais"""
    positions = self.position_list_widget.get_all_positions()

    if not positions:
        QMessageBox.warning(self, "Aviso", "Adicione pelo menos uma posição")
        return

    sequence_name = self.sequence_widget.sequence_name.text()
    if not sequence_name:
        QMessageBox.warning(self, "Aviso", "Digite um nome para a sequência")
        return

    self.current_sequence = self.controller.create_sequence(sequence_name, positions)
    self.statusBar().showMessage(f'Sequência '{sequence_name}' criada com {{len(positions)}} posições")
    self.sequence_widget.sequence_status.setText(f'Criada: {{len(positions)}} posições")

```

```

def run_sequence(self):
    """Executes the current sequence"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Warning", "Create a sequence first")
        return

    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Warning", "CNC not connected")
        return

    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Warning", "Camera not connected")
        return

    # Clear previous results
    self.results_table.setRowCount(0)

    # Configure UI for execution
    self.is_running_sequence = True
    self.sequence_widget.run_sequence_btn.setEnabled(False)
    self.sequence_widget.stop_sequence_btn.setEnabled(True)
    self.sequence_widget.sequence_status.setText("Executing...")

    # Start execution
    try:
        self.statusBar().showMessage(f"Executing sequence '{self.current_sequence.name}'...")

        # Use a thread to run the sequence
        self.run_thread = SequenceRunnerThread(self.controller, self.current_sequence.name)
        self.run_thread.image_captured.connect(self.on_sequence_image_captured)
        self.run_thread.sequence_completed.connect(self.on_sequence_completed)
        self.run_thread.sequence_error.connect(self.on_sequence_error)
        self.run_thread.start()

    except Exception as e:
        self.statusBar().showMessage(f"Error executing sequence: {str(e)}")
        self.sequence_widget.sequence_status.setText("Error")
        self.on_sequence_completed()

def on_sequence_image_captured(self, result):
    """Called when an image is captured during sequence execution"""
    position = result["position"]
    image = result["image"]
    timestamp = result["timestamp"]

    # Display the image
    self.image_viewer.display_image(image, f"Position: {position.name} ({position.x:.3f}, {position.y:.3f})")

    # Add to results table
    row = self.results_table.rowCount()

```

```
self.results_table.insertRow(row)
self.results_table.setItem(row, 0, QTableWidgetItem(position.name))
self.results_table.setItem(row, 1, QTableWidgetItem(time.strftime("%H:%M:%S", time.localtime(timestamp))))
self.results_table.setItem(row, 2, QTableWidgetItem("Captured"))
```

```
def stop_sequence(self):
    """Para a execução da sequência atual"""
    if self.is_running_sequence:
        self.controller.stop_sequence()
        self.statusBar().showMessage("Execução de sequência interrompida")
        self.on_sequence_completed()
```

```
def on_sequence_completed(self):
    """Chamado quando a execução da sequência é concluída"""
    self.is_running_sequence = False
    self.sequence_widget.run_sequence_btn.setEnabled(True)
    self.sequence_widget.stop_sequence_btn.setEnabled(False)
    self.sequence_widget.sequence_status.setText("Concluída")
    self.statusBar().showMessage("Execução de sequência concluída")
```

```
def on_sequence_error(self, error_message):
    """Chamado quando ocorre um erro durante a execução da sequência"""
    QMessageBox.critical(self, "Erro na Sequência", error_message)
    self.on_sequence_completed()
```

```
def save_program(self):
    """Salva o programa de inspeção atual"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return
```

```
filename, _ = QFileDialog.getSaveFileName(self, "Salvar Programa", "", "Arquivos JSON (*.json)")
```

```
if filename:
    if not filename.endswith('.json'):
        filename += '.json'

    if self.controller.save_positions(filename):
        self.statusBar().showMessage(f"Programa salvo em {filename}")
    else:
        QMessageBox.critical(self, "Erro", "Falha ao salvar o programa")
```

```
def load_program(self):
    """Carrega um programa de inspeção salvo"""
    filename, _ = QFileDialog.getOpenFileName(self, "Carregar Programa", "", "Arquivos JSON (*.json)")

    if filename:
        if self.controller.load_positions(filename):
            # Atualiza a interface
            self.position_list_widget.clear_positions()
```

```

# Adiciona posições carregadas à lista
for name, position in self.controller.position_manager.positions.items():
    self.position_list_widget.add_position(position)

# Se há sequências, usa a primeira como atual
if self.controller.position_manager.sequences:
    sequence_name = next(iter(self.controller.position_manager.sequences))
    self.current_sequence = self.controller.position_manager.sequences[sequence_name]
    self.sequence_widget.sequence_name.setText(sequence_name)
    self.sequence_widget.sequence_status.setText(f"Carregada: {len(self.current_sequence.positions)} posições")

self.statusBar().showMessage(f"Programa carregado de {filename}")
else:
    QMessageBox.critical(self, "Erro", "Falha ao carregar o programa")

```

```

def closeEvent(self, event):
    """Manipula o evento de fechamento da janela"""
    # Para a execução da sequência, se houver
    if self.is_running_sequence:
        self.controller.stop_sequence()

    # Desconecta da câmera e CNC
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:
        self.controller.camera.disconnect()

    if self.controller.cnc.is_connected:
        self.controller.cnc.disconnect()

    event.accept()

```

```

class SequenceRunnerThread(QThread):
    """Thread para executar uma sequência de inspeção"""
    image_captured = pyqtSignal(dict) # Emite resultados da captura
    sequence_completed = pyqtSignal() # Emite quando a sequência é concluída
    sequence_error = pyqtSignal(str) # Emite quando ocorre um erro

    def __init__(self, controller, sequence_name):
        super().__init__()
        self.controller = controller
        self.sequence_name = sequence_name

    def run(self):
        try:
            # Execute a sequência e processe os resultados
            def process_result(result):
                self.image_captured.emit(result)

            self.controller.run_sequence(self.sequence_name, process_result)
            self.sequence_completed.emit()

        except Exception as e:

```

```

        self.sequence_error.emit(str(e))

# Classe QInputDialog que estava faltando
from PyQt6.QtWidgets import QInputDialog

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = AOIControllerApp()
    window.show()
    sys.exit(app.exec())

```

File: controlador.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi

```

import sys
import serial
import serial.tools.list_ports
import time
import re
import threading
import logging
from queue import Queue, Empty, PriorityQueue
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QPushButton, QLabel, QSlider, QGroupBox,
                             QGridLayout, QLineEdit, QComboBox, QProgressBar,
                             QFrame, QPushButton)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer, QObject
from PyQt6.QtGui import QFont, QColor, QPalette, QIcon

# Importa a biblioteca grbl-streamer
from grbl_streamer import GrblStreamer

# Configuração de logging para diagnóstico
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("grbl_comm.log"),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger("GRBLComm")

class GRBLCommunicationThread(QThread):
    """Thread para lidar com a comunicação com GRBL sem bloquear a interface"""
    response_received = pyqtSignal(str)
    status_update = pyqtSignal(str)
    position_update = pyqtSignal(dict)
    error_message = pyqtSignal(str)

```

```

def __init__(self, parent=None):
    super().__init__(parent)
    self.serial_port = None
    self.is_connected = False
    self.running = True
    # Usando PriorityQueue como em consumo_lib.py
    self.command_queue = PriorityQueue()
    # Usando um único lock para simplicidade
    self.lock = threading.Lock()
    self.is_moving = False
    self.last_status_check = 0
    self.last_position = {'x': 0, 'y': 0, 'z': 0}
    self.machine_state = "Unknown"

    logger.info("GRBLCommunicationThread inicializada")

def set_serial_port(self, serial_port):
    """Define a porta serial para comunicação."""
    logger.info(f"Definindo porta serial: {serial_port}")
    self.serial_port = serial_port
    self.is_connected = True

def disconnect(self):
    """Desconecta a porta serial."""
    logger.info("Desconectando porta serial")
    self.is_connected = False
    if self.serial_port and self.serial_port.is_open:
        try:
            with self.lock:
                logger.debug("Fechando porta serial")
                self.serial_port.close()
                logger.info("Porta serial fechada com sucesso")
        except Exception as e:
            logger.error(f"Erro ao fechar porta serial: {e}")

def send_command(self, command, priority=False):
    """
    Adiciona um comando à fila de execução, com opção de prioridade.

    Args:
        command: O comando GCODE
        priority: Se True, o comando tem prioridade
    """
    # Comandos de controle imediato (!, ~, ?) são sempre processados imediatamente
    if command in ['!', '~', '?']:
        priority_level = -1 # Prioridade máxima
    else:
        priority_level = 0 if priority else 1

    logger.debug(f"Enviando comando: '{command}' (priority={priority_level})")

```

```
self.command_queue.put((priority_level, command))
```

```
def run(self):
```

```
    """Método principal da thread - processa os comandos e monitora o status"""
```

```
    logger.info("Thread de comunicação iniciada")
```

```
    last_diagnostic_time = 0
```

```
    last_command_time = 0
```

```
    while self.running:
```

```
        try:
```

```
            # Diagnóstico periódico
```

```
            current_time = time.time()
```

```
            if current_time - last_diagnostic_time > 10:
```

```
                if self.is_connected and self.serial_port and self.serial_port.is_open:
```

```
                    buffer_size = self.serial_port.in_waiting if hasattr(self.serial_port, 'in_waiting') else 0
```

```
                    logger.info(f"Diagnóstico de comunicação: Conectado, buffer_in={buffer_size}, is_moving={self.is_moving}, e
```

```
                # Limpar buffer se estiver muito grande
```

```
                if buffer_size > 200:
```

```
                    logger.warning(f"Buffer muito grande ({buffer_size} bytes), limpando...")
```

```
                    with self.lock:
```

```
                        self.serial_port.reset_input_buffer()
```

```
                else:
```

```
                    logger.info(f"Diagnóstico de comunicação: Desconectado, is_moving={self.is_moving}, estado={self.machine
```

```
                    last_diagnostic_time = current_time
```

```
            # Verificar conexão
```

```
            if not self.is_connected or not self.serial_port or not self.serial_port.is_open:
```

```
                logger.debug("Não conectado, aguardando...")
```

```
                time.sleep(0.1)
```

```
                continue
```

```
            # Verificar e processar dados recebidos mesmo sem enviar comando
```

```
            if self.serial_port.in_waiting > 0:
```

```
                with self.lock:
```

```
                    data = self.serial_port.readline().decode('utf-8', errors='replace').strip()
```

```
            if data:
```

```
                logger.debug(f"Dados recebidos: '{data}'")
```

```
            # Processar dados recebidos
```

```
            if data.startswith('<') and '|' in data:
```

```
                self._parse_status(data)
```

```
            elif "ok" in data or "error" in data:
```

```
                self.response_received.emit(data)
```

```
            # Limita a taxa de envio de comandos
```

```
            can_send_command = True
```

```
            if current_time - last_command_time < 0.1: # Máximo 10 comandos por segundo
```

```
                can_send_command = False
```

```

# Processa comandos na fila
if can_send_command and not self.command_queue.empty():
    priority, command = self.command_queue.get(block=False)

    # Comandos prioritários ou de controle podem sempre ser enviados
    if priority <= 0 or command in ['!', '~', '?']:
        pass # Sempre permite
    # Comandos regulares verificam o estado da máquina
    elif self.is_moving and "Hold" not in self.machine_state:
        logger.debug(f"Máquina em movimento, enfileirando comando: {command}")
        self.command_queue.put((priority, command))
        self.command_queue.task_done()
        time.sleep(0.05)
        continue

    # Preparar comando
    if not command.endswith('\n'):
        command += '\n'

    # Identificar tipo de comando
    is_movement_cmd = any(cmd in command for cmd in ['G0', 'G1', 'G2', 'G3'])

    # Enviar comando
    with self.lock:
        logger.debug(f"Enviando bytes: {command.encode()}")
        self.serial_port.write(command.encode())
        self.serial_port.flush()

    # Se for comando de movimento, atualiza estado
    if is_movement_cmd:
        logger.info("Atualizando flag is_moving para True")
        self.is_moving = True

    # Atualiza timestamp
    last_command_time = current_time

    # Para comandos rápidos (controle), não espera resposta
    if command.strip() in ['!', '~', '?']:
        self.command_queue.task_done()
        continue

    # Para outros comandos, espera resposta
    start_time = time.time()
    response = ""

    # Lê respostas com timeout curto (200ms)
    while (time.time() - start_time) < 0.2:
        if self.serial_port.in_waiting > 0:
            line = self.serial_port.readline().decode('utf-8', errors='replace').strip()
            if line:
                if not line.startswith('<'):

```

```

        response += line + " "
        # Se recebeu ok ou error, finaliza
        if "ok" in line or "error" in line:
            break
        # Status atualiza estado mas não é considerado resposta final
        elif line.startswith('<') and '|' in line:
            self._parse_status(line)
    else:
        time.sleep(0.01)

    # Finaliza comando
    self.command_queue.task_done()

    # Processa resposta
    if response:
        self.response_received.emit(response.strip())
        if is_movement_cmd and "error" in response.lower():
            logger.warning(f"Erro GRBL em comando de movimento: {response}")
            self.error_message.emit(f"Erro GRBL: {response}")
            self.is_moving = False

    # Verificação periódica de status
    if (current_time - self.last_status_check) > 0.2:
        self._check_status_simple()
        self.last_status_check = current_time

    # Pausa pequena para não consumir CPU
    time.sleep(0.01)

except Exception as e:
    logger.error(f"Erro na thread de comunicação: {e}", exc_info=True)
    self.error_message.emit(f"Erro na thread de comunicação: {str(e)}")
    time.sleep(0.1)

def _check_status_simple(self):
    """Verifica o status atual do GRBL (implementação simplificada)"""
    try:
        if not self.is_connected or not self.serial_port or not self.serial_port.is_open:
            return

        # Apenas envia o comando de status
        with self.lock:
            logger.debug("Enviando comando de status '?'")
            self.serial_port.write(b"?\n")
            self.serial_port.flush()
            # Não aguardamos resposta aqui, ela será processada no próximo ciclo

    except Exception as e:
        logger.error(f"Erro ao verificar status: {e}")

# Modifique o método _parse_status na classe GRBLCommunicationThread

```

```
def _parse_status(self, status_line):
```

```
    """
```

Analisa a linha de status do GRBL.

Formato típico: <Idle|MPos:0.000,0.000,0.000|FS:0,0|WCO:0.000,0.000,0.000>

```
    """
```

```
try:
```

```
    logger.debug(f"Analisando status: '{status_line}'")
```

```
    # Extrai o estado da máquina
```

```
    if status_line.startswith('<') and '|' in status_line:
```

```
        state = status_line[1:status_line.find('|')]
```

```
        old_state = self.machine_state
```

```
        self.machine_state = state
```

```
        logger.debug(f"Estado da máquina: {old_state} -> {state}")
```

```
    # Atualiza a flag de movimento com base no estado
```

```
    old_moving = self.is_moving
```

```
    # Estados onde a máquina está definitivamente parada
```

```
    if state in ["Idle", "Alarm", "Error", "Door", "Check", "Home"]:
```

```
        self.is_moving = False
```

```
    # Estados onde a máquina está definitivamente em movimento
```

```
    elif state in ["Run", "Jog"]:
```

```
        self.is_moving = True
```

```
    # Estado Hold - a máquina está temporariamente parada, mas não completamente Idle
```

```
    elif state.startswith("Hold"):
```

```
        # Para os nossos propósitos, consideramos Hold como "não está em movimento"
```

```
        # Isso permite que novos comandos sejam enviados após liberar o Hold
```

```
        self.is_moving = False
```

```
    if old_moving != self.is_moving:
```

```
        logger.info(f"Flag is_moving atualizada: {old_moving} -> {self.is_moving}")
```

```
    # Emite estado para atualizar UI quando o estado muda
```

```
    if old_state != state:
```

```
        self.status_update.emit(status_line)
```

```
    # Extrai a posição
```

```
    match = re.search(r'(?:(MPos|WPos):(-?\d+\.\d+),(-?\d+\.\d+),(-?\d+\.\d+))', status_line)
```

```
    if match:
```

```
        x, y, z = match.groups()
```

```
        position = {
```

```
            'x': float(x),
```

```
            'y': float(y),
```

```
            'z': float(z)
```

```
        }
```

```
    # Verifica se houve mudança significativa na posição para log
```

```
    position_changed = (
```

```
        abs(position['x'] - self.last_position['x']) > 0.001 or
```

```

        abs(position['y'] - self.last_position['y']) > 0.001 or
        abs(position['z'] - self.last_position['z']) > 0.001
    )

```

```

    # Se houve mudança significativa, registra no log
    if position_changed:
        logger.info(f"POSIÇÃO ALTERADA: {self.last_position} → {position}")

```

```

    # Sempre emite a atualização e atualiza a última posição
    self.position_update.emit(position)
    self.last_position = position

```

```

except Exception as e:
    logger.error(f"Erro ao analisar status: {e}", exc_info=True)

```

```

class SignalEmitter(QObject):
    """Classe para emitir sinais Qt de dentro da biblioteca grbl-streamer"""
    response_received = pyqtSignal(str)
    status_update = pyqtSignal(dict)
    error_message = pyqtSignal(str)

```

```

class DirectionalButton(QPushButton):
    """Botão direcional com setas para controle intuitivo de movimento"""
    def __init__(self, direction, parent=None):
        super().__init__(parent)
        self.direction = direction

    # Configura a aparência do botão
    self.setMinimumSize(60, 60)
    self.setMaximumSize(60, 60)

    # Define texto/símbolo com base na direção
    if direction == "up":
        self.setText("↑")
    elif direction == "down":
        self.setText("↓")
    elif direction == "left":
        self.setText("←")
    elif direction == "right":
        self.setText("→")

    # Estiliza o botão para parecer com botão de jog
    font = QFont()
    font.setBold(True)
    font.setPointSize(20)
    self.setFont(font)

```

```

class JogButton(QPushButton):
    """Botão personalizado para movimentos jog"""
    def __init__(self, text, parent=None):
        super().__init__(text, parent)

```

```
# Estilização
self.setMinimumSize(60, 60)
font = QFont()
font.setBold(True)
font.setPointSize(12)
self.setFont(font)
```

```
class PositionDisplay(QFrame):
    """Widget para exibir a posição atual da máquina"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setFrameShape(QFrame.Shape.StyledPanel)
        self.setFrameShadow(QFrame.Shadow.Raised)

        layout = QGridLayout(self)

        # Rótulos para cabeçalhos
        title_font = QFont()
        title_font.setBold(True)

        title_label = QLabel("Posição Atual:")
        title_label.setFont(title_font)
        layout.addWidget(title_label, 0, 0, 1, 2)

        layout.addWidget(QLabel("X:"), 1, 0)
        layout.addWidget(QLabel("Y:"), 2, 0)

        # Valores de posição
        self.x_pos = QLabel("0.000")
        self.y_pos = QLabel("0.000")

        pos_font = QFont("Monospace")
        pos_font.setPointSize(12)
        pos_font.setBold(True)

        self.x_pos.setFont(pos_font)
        self.y_pos.setFont(pos_font)

        self.x_pos.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)
        self.y_pos.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)

        layout.addWidget(self.x_pos, 1, 1)
        layout.addWidget(self.y_pos, 2, 1)

    def update_position(self, position):
        """Atualiza os valores de posição exibidos"""
        self.x_pos.setText(f"{position.get('x', 0):.3f} mm")
        self.y_pos.setText(f"{position.get('y', 0):.3f} mm")

class JogController:
```

"""Controlador dedicado para operações de jogging"""

```
def __init__(self, comm_thread):
    self.comm_thread = comm_thread
    self.is_jogging = False
    self.current_direction = None
    self.current_axis = None
    self.jog_timer = QTimer()
    self.jog_timer.timeout.connect(self._execute_jog_step)
    self.jog_step_size = 1.0 # Aumentado para 1mm por passo para movimento mais visível
    self.jog_speed = 1000 # mm/min
    self.last_command_time = 0
    self.command_sent = False
```

```
def start_jog(self, axis, direction):
    """Inicia o modo de jogging contínuo"""
    logger.debug(f'JogController: iniciando jog {axis}{'+' if direction > 0 else '-'}')

    # Se já estiver em jog com os mesmos parâmetros, não reinicia
    if self.is_jogging and self.current_axis == axis and self.current_direction == direction:
        return
```

```
    # Para qualquer jog anterior antes de iniciar um novo
    self.stop_jog()
```

```
    # Configura o novo movimento
    self.is_jogging = True
    self.current_axis = axis
    self.current_direction = direction
    self.command_sent = False
```

```
    # Inicia o processo de jog com base no estado atual da máquina
    if "Hold" in self.comm_thread.machine_state:
        logger.debug("JogController: máquina em Hold, enviando comando de retomada")
        self.comm_thread.send_command("~", priority=True)
        # Aguarda para dar tempo ao comando de retomada ser processado
        QTimer.singleShot(100, self._prepare_for_jog)
    else:
        self._prepare_for_jog()
```

```
def _prepare_for_jog(self):
    """Prepara a máquina para realizar um movimento de jog"""
    # Garante modo relativo
    self.comm_thread.send_command("G91", priority=True)

    # Espera um curto período para garantir que o comando seja processado
    QTimer.singleShot(50, self._start_jog_timer)
```

```
def _start_jog_timer(self):
    """Inicia o timer que controla o movimento de jog"""
    if not self.is_jogging:
```

```

        return # Verifica se o jog ainda está ativo

# Executa o primeiro movimento imediatamente
self._execute_jog_step()

# Configura o timer para repetir o movimento periodicamente
# Intervalo maior para melhor estabilidade
self.jog_timer.start(300) # 300ms entre passos

def _execute_jog_step(self):
    """Executa um único passo de jog"""
    if not self.is_jogging:
        return

    # Impede o envio de comandos muito frequentes
    current_time = time.time()
    if current_time - self.last_command_time < 0.2:
        return

    # Verifica condições de estado da máquina
    if "Hold" in self.comm_thread.machine_state:
        logger.debug("JogController: detectado estado Hold durante movimento, enviando retomada")
        self.comm_thread.send_command("~", priority=True)
        return

    # Calcula a distância do movimento
    distance = self.current_direction * self.jog_step_size

    # Cria o comando de movimento
    if self.current_axis == "X":
        command = f"G0 X{distance} F{self.jog_speed}"
    else: # assume "Y"
        command = f"G0 Y{distance} F{self.jog_speed}"

    logger.debug(f"JogController: enviando comando {command}")

    # Envia o comando com alta prioridade
    self.comm_thread.send_command(command, priority=True)
    self.last_command_time = current_time
    self.command_sent = True

def stop_jog(self):
    """Para o movimento jog"""
    if not self.is_jogging:
        return

    logger.debug("JogController: parando jog")

    # Interrompe o timer
    self.jog_timer.stop()
    self.is_jogging = False

```

```
# Envia comando de parada apenas se um comando foi enviado antes
if self.command_sent:
    # Envia feed hold para parar movimento
    self.comm_thread.send_command("!", priority=True)

    # Após uma pequena pausa, envia comando de retomada para liberar máquina
    QTimer.singleShot(150, self._send_resume_after_stop)
```

```
def _send_resume_after_stop(self):
    """Envia comando de retomada após parada para liberar a máquina"""
    # Envio do comando "~" para liberar máquina do estado Hold
    self.comm_thread.send_command("~", priority=True)

    # Solicita status atualizado
    QTimer.singleShot(50, lambda: self.comm_thread.send_command("?", priority=True))
```

```
class GRBLController(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Controle CNC GRBL Avançado")
        self.setGeometry(100, 100, 1000, 700)

        # Inicialização do sistema de comunicação
        self.signals = SignalEmitter()
        self.grbl = None
        self.is_connected = False
        self.current_position = {'x': 0, 'y': 0, 'z': 0}
        self.machine_status = "Desconectado"
        self.last_command = ""
        self.last_response = ""
        self.last_error = ""

        # Controle de jog contínuo
        self.jogging = False
        self.current_jog_axis = None
        self.current_jog_direction = None

        # Estado da máquina
        self.current_motion_mode = "G91" # Começa em modo relativo

        # Conectar os sinais
        self.signals.response_received.connect(self.on_response_received)
        self.signals.status_update.connect(self.on_status_update)
        self.signals.error_message.connect(self.on_error_message)

        # Configuração da interface
        self.init_ui()

        # Timer para atualizar o status da interface
```

```
self.ui_timer = QTimer(self)
self.ui_timer.timeout.connect(self.update_ui_state)
self.ui_timer.start(300) # Atualiza a cada 300ms
```

```
def init_ui(self):
```

```
    # Widget central
```

```
    central_widget = QWidget()
```

```
    self.setCentralWidget(central_widget)
```

```
    # Layout principal
```

```
    main_layout = QVBoxLayout(central_widget)
```

```
    # Grupo de conexão
```

```
    connection_group = QGroupBox("Conexão Arduino")
```

```
    connection_layout = QHBoxLayout()
```

```
    self.port_combo = QComboBox()
```

```
    self.connect_button = QPushButton("Conectar")
```

```
    self.connect_button.clicked.connect(self.toggle_connection)
```

```
    self.refresh_button = QPushButton("Atualizar Portas")
```

```
    self.refresh_button.clicked.connect(self.refresh_ports)
```

```
    connection_layout.addWidget(QLabel("Porta:"))
```

```
    connection_layout.addWidget(self.port_combo)
```

```
    connection_layout.addWidget(self.connect_button)
```

```
    connection_layout.addWidget(self.refresh_button)
```

```
    connection_group.setLayout(connection_layout)
```

```
    main_layout.addWidget(connection_group)
```

```
    # Layout horizontal para controles e status
```

```
    control_status_layout = QHBoxLayout()
```

```
    # Grupo de controle dos eixos (lado esquerdo)
```

```
    axis_group = QGroupBox("Controle de Eixos")
```

```
    axis_layout = QGridLayout()
```

```
    # Controles do eixo X
```

```
    axis_layout.addWidget(QLabel("Eixo X:"), 0, 0, Qt.AlignmentFlag.AlignRight)
```

```
    x_control_layout = QHBoxLayout()
```

```
    self.x_minus_button = JogButton("X-")
```

```
    self.x_minus_button.pressed.connect(lambda: self.start_jog("X", -1))
```

```
    self.x_minus_button.released.connect(self.stop_jog)
```

```
    self.x_plus_button = JogButton("X+")
```

```
    self.x_plus_button.pressed.connect(lambda: self.start_jog("X", 1))
```

```
    self.x_plus_button.released.connect(self.stop_jog)
```

```

self.x_distance = QLineEdit("1")
self.x_distance.setAlignment(Qt.AlignmentFlag.AlignCenter)

x_control_layout.addWidget(self.x_minus_button)
x_control_layout.addWidget(self.x_distance)
x_control_layout.addWidget(self.x_plus_button)

axis_layout.addLayout(x_control_layout, 0, 1)

# Controles do eixo Y
axis_layout.addWidget(QLabel("Eixo Y:"), 1, 0, Qt.AlignmentFlag.AlignRight)

y_control_layout = QHBoxLayout()
self.y_minus_button = JogButton("Y-")
self.y_minus_button.pressed.connect(lambda: self.start_jog("Y", -1))
self.y_minus_button.released.connect(self.stop_jog)

self.y_plus_button = JogButton("Y+")
self.y_plus_button.pressed.connect(lambda: self.start_jog("Y", 1))
self.y_plus_button.released.connect(self.stop_jog)

self.y_distance = QLineEdit("1")
self.y_distance.setAlignment(Qt.AlignmentFlag.AlignCenter)

y_control_layout.addWidget(self.y_minus_button)
y_control_layout.addWidget(self.y_distance)
y_control_layout.addWidget(self.y_plus_button)

axis_layout.addLayout(y_control_layout, 1, 1)

# Controle direcional em formato de teclado numérico (8, 4, 6, 2)
directional_group = QGroupBox("Controle Direcional")
directional_layout = QGridLayout()

self.dir_up_button = DirectionalButton("up")
self.dir_left_button = DirectionalButton("left")
self.dir_right_button = DirectionalButton("right")
self.dir_down_button = DirectionalButton("down")

# Conectar eventos de pressionar/soltar para movimentos contínuos
self.dir_up_button.pressed.connect(lambda: self.start_continuous_jog("Y", 1))
self.dir_up_button.released.connect(self.stop_continuous_jog)

self.dir_down_button.pressed.connect(lambda: self.start_continuous_jog("Y", -1))
self.dir_down_button.released.connect(self.stop_continuous_jog)

self.dir_left_button.pressed.connect(lambda: self.start_continuous_jog("X", -1))
self.dir_left_button.released.connect(self.stop_continuous_jog)

self.dir_right_button.pressed.connect(lambda: self.start_continuous_jog("X", 1))
self.dir_right_button.released.connect(self.stop_continuous_jog)

```

```

# Organização em grade para parecer com teclado numérico
directional_layout.addWidget(self.dir_up_button, 0, 1)
directional_layout.addWidget(self.dir_left_button, 1, 0)
directional_layout.addWidget(self.dir_right_button, 1, 2)
directional_layout.addWidget(self.dir_down_button, 2, 1)

# Jog step - valor incremental para os botões direcionais
step_layout = QHBoxLayout()
step_layout.addWidget(QLabel("Passo:"))
self.jog_step = QLineEdit("1")
self.jog_step.setAlignment(Qt.AlignmentFlag.AlignCenter)
step_layout.addWidget(self.jog_step)
step_layout.addWidget(QLabel("mm"))

directional_layout.addLayout(step_layout, 3, 0, 1, 3)
directional_group.setLayout(directional_layout)

# Adiciona o grupo direcional ao layout principal
axis_layout.addWidget(directional_group, 2, 0, 2, 2)

# Velocidade de movimento (feed rate)
axis_layout.addWidget(QLabel("Velocidade:"), 4, 0, Qt.AlignmentFlag.AlignRight)

feed_layout = QHBoxLayout()
self.feed_rate = QLineEdit("1000")
self.feed_rate.setAlignment(Qt.AlignmentFlag.AlignCenter)

# Slider para controle de velocidade
self.feed_slider = QSlider(Qt.Orientation.Horizontal)
self.feed_slider.setMinimum(100)
self.feed_slider.setMaximum(5000)
self.feed_slider.setValue(1000)
self.feed_slider.setTickPosition(QSlider.TickPosition.TicksBelow)
self.feed_slider.setTickInterval(500)

# Conectar slider e campo de texto
self.feed_slider.valueChanged.connect(self.update_feed_rate)
self.feed_rate.textChanged.connect(self.update_feed_slider)

feed_layout.addWidget(self.feed_rate)
feed_layout.addWidget(QLabel("mm/min"))
feed_layout.addWidget(self.feed_slider)

axis_layout.addLayout(feed_layout, 4, 1)

# Modos de movimento
axis_layout.addWidget(QLabel("Modo:"), 5, 0, Qt.AlignmentFlag.AlignRight)

mode_layout = QHBoxLayout()
self.mode_absolute = QPushButton("Passo a Passo (G90)")

```

```

self.mode_absolute.setCheckable(True)
self.mode_absolute.clicked.connect(lambda: self.set_motion_mode("G90"))

self.mode_relative = QPushButton("Contínuo (G91)")
self.mode_relative.setCheckable(True)
self.mode_relative.setChecked(True) # Começa em modo relativo
self.mode_relative.clicked.connect(lambda: self.set_motion_mode("G91"))

mode_layout.addWidget(self.mode_absolute)
mode_layout.addWidget(self.mode_relative)

axis_layout.addLayout(mode_layout, 5, 1)

# Botões de controle da máquina
control_buttons_layout = QGridLayout()

self.home_button = QPushButton("Home ($H)")
self.home_button.clicked.connect(self.home)

self.unlock_button = QPushButton("Desbloquear ($X)")
self.unlock_button.clicked.connect(self.unlock)

self.zero_button = QPushButton("Zerar Posição (G92)")
self.zero_button.clicked.connect(self.set_zero)

self.reset_button = QPushButton("Reset Alarme")
self.reset_button.setStyleSheet("background-color: orange; color: white; font-weight: bold;")
self.reset_button.clicked.connect(self.reset_alarm)

self.stop_button = QPushButton("PARAR (!)")
self.stop_button.setStyleSheet("background-color: red; color: white; font-weight: bold;")
self.stop_button.clicked.connect(self.emergency_stop)

control_buttons_layout.addWidget(self.home_button, 0, 0)
control_buttons_layout.addWidget(self.unlock_button, 0, 1)
control_buttons_layout.addWidget(self.zero_button, 1, 0)
control_buttons_layout.addWidget(self.reset_button, 1, 1) # Novo botão de reset
control_buttons_layout.addWidget(self.stop_button, 2, 0, 1, 2) # Ocupa duas colunas

axis_layout.addLayout(control_buttons_layout, 6, 0, 1, 2)

# Console de comandos
axis_layout.addWidget(QLabel("Comando GCODE:"), 7, 0, 1, 2)

command_layout = QHBoxLayout()
self.command_input = QLineEdit()
self.command_input.returnPressed.connect(self.send_command)

self.send_button = QPushButton("Enviar")
self.send_button.clicked.connect(self.send_command)

```

```
command_layout.addWidget(self.command_input)
command_layout.addWidget(self.send_button)

axis_layout.addLayout(command_layout, 8, 0, 1, 2)

axis_group.setLayout(axis_layout)
control_status_layout.addWidget(axis_group)

# Painel de status (lado direito)
status_group = QGroupBox("Status da Máquina")
status_layout = QVBoxLayout()

# Widget de exibição de posição
self.position_display = PositionDisplay()
status_layout.addWidget(self.position_display)

# Status da máquina
status_layout.addWidget(QLabel("Estado:"))
self.machine_state_label = QLabel("Desconectado")
self.machine_state_label.setFont(QFont("Arial", 12, QFont.Weight.Bold))
status_layout.addWidget(self.machine_state_label)

# Últimos comandos e respostas
status_layout.addWidget(QLabel("Último Comando:"))
self.last_command_label = QLabel("-")
status_layout.addWidget(self.last_command_label)

status_layout.addWidget(QLabel("Última Resposta:"))
self.last_response_label = QLabel("-")
status_layout.addWidget(self.last_response_label)

# Log de comunicação
status_layout.addWidget(QLabel("Log de Comunicação:"))
self.comm_log = QLineEdit()
self.comm_log.setReadOnly(True)
status_layout.addWidget(self.comm_log)

# Otimizações de comunicação
optimization_group = QGroupBox("Otimizações")
optimization_layout = QVBoxLayout()

self.fast_mode_check = QPushButton("Modo Rápido (Sem Door Check)")
self.fast_mode_check.setCheckable(True)
self.fast_mode_check.clicked.connect(self.toggle_fast_mode)
self.fast_mode_check.setToolTip("Desativa verificação de porta para movimentos mais rápidos")

optimization_layout.addWidget(self.fast_mode_check)
optimization_group.setLayout(optimization_layout)
status_layout.addWidget(optimization_group)

status_layout.addStretch()
```

```

status_group.setLayout(status_layout)
control_status_layout.addWidget(status_group)

# Proporção entre painéis de controle e status
control_status_layout.setStretch(0, 3)
control_status_layout.setStretch(1, 2)

main_layout.addLayout(control_status_layout)

# Barra de status
self.statusBar = self.statusBar()
self.statusBar.showMessage("Pronto para conexão")

# Desabilitar controles até que a conexão seja estabelecida
self.toggle_controls(False)

# Atualizar portas
self.refresh_ports()

def reset_alarm(self):
    """Reset o estado de alarme e libera a máquina para movimento"""
    if not self.is_connected or not self.grbl:
        return

    try:
        logger.debug("Resetando alarme")

        # Sequência completa de reset para garantir que o alarme seja cancelado
        # 1. Envia comando kill alarm
        self.grbl.send_immediately("$X")

        # 2. Pausa breve e mais comandos para garantir a recuperação
        QTimer.singleShot(150, lambda: self.grbl.send_immediately("~")) # Resume

        # 3. Restaura o modo de movimento atual
        QTimer.singleShot(300, lambda: self.grbl.send_immediately(self.current_motion_mode))

        # 4. Soft reset para caso de falha dos comandos acima
        QTimer.singleShot(400, lambda: self.grbl.send_immediately("\x18")) # Ctrl+X (soft reset)

        # 5. Restaura novamente o modo após soft reset
        QTimer.singleShot(600, lambda: self.grbl.send_immediately(self.current_motion_mode))

        self.statusBar.showMessage("Reset de alarme enviado, máquina deve estar pronta em breve")
    except Exception as e:
        logger.error(f"Erro ao resetar alarme: {str(e)}", exc_info=True)
        self.statusBar.showMessage(f"Erro: {str(e)}")

def refresh_ports(self):
    """Detecta automaticamente as portas seriais disponíveis"""
    self.port_combo.clear()

```

```

available_ports = [port.device for port in serial.tools.list_ports.comports()]

if not available_ports:
    self.statusBar.showMessage("Nenhuma porta serial encontrada")
    return

self.port_combo.addItems(available_ports)

# Verificar se COM9 está na lista e selecioná-la
com9_index = self.port_combo.findText("COM9")
if com9_index >= 0:
    self.port_combo.setCurrentIndex(com9_index)
    self.statusBar.showMessage("Arduino detectado na porta COM9")
else:
    self.statusBar.showMessage(f"Portas encontradas: {' '.join(available_ports)}")

def toggle_connection(self):
    """Conecta/desconecta da máquina CNC"""
    if not self.is_connected:
        try:
            port = self.port_combo.currentText()
            if not port:
                self.statusBar.showMessage("Nenhuma porta selecionada")
                return

            # Mensagem de status
            self.statusBar.showMessage(f"Conectando à porta {port}...")

            # CORREÇÃO: Inicializar a comunicação com o GRBL usando a API correta da biblioteca
            def grbl_callback(eventstring, *data):
                """Callback para eventos do GrblStreamer"""
                logger.debug(f"GRBL evento: {eventstring}, dados: {data}")

            # Processar eventos específicos
            if eventstring == "on_stateupdate":
                # Formato: on_stateupdate(state, mpos, wpos)
                if len(data) >= 3:
                    state = data[0]
                    machine_pos = data[1] # (x, y, z)

                    # Atualiza estado da máquina
                    self.machine_status = state

                    # Atualiza posição
                    if len(machine_pos) >= 3:
                        self.current_position = {
                            'x': machine_pos[0],
                            'y': machine_pos[1],
                            'z': machine_pos[2] if len(machine_pos) > 2 else 0
                        }

```

```

        self.signals.status_update.emit(self.current_position)

    elif eventstring == "on_processed_command":
        # Respostas a comandos
        if len(data) >= 2:
            response = data[1]
            self.signals.response_received.emit(str(response))

    elif eventstring == "on_error":
        # Mensagens de erro
        if data:
            error_msg = str(data[0])
            self.signals.error_message.emit(error_msg)

# Inicializa com o callback
self.grbl = GrblStreamer(grbl_callback)

# Configura logging (opcional)
self.grbl.setup_logging()

# Conecta usando o método correto
self.grbl.connect(port, 115200)

# Desbloqueia a máquina
self.grbl.send_immediately("$X")

# Iniciar em modo relativo para jog
self.grbl.send_immediately("G91")
self.mode_relative.setChecked(True)
self.mode_absolute.setChecked(False)

# Inicia verificação de status
self.grbl.poll_start()

# Atualizar o estado da interface
self.is_connected = True
self.machine_status = "Idle" # Estado inicial presumido
self.connect_button.setText("Desconectar")
self.machine_state_label.setText("Conectado")
self.statusBar.showMessage(f"Conectado ao GRBL na porta {port}")
self.toggle_controls(True)

except Exception as e:
    logger.error(f"Erro ao conectar: {str(e)}", exc_info=True)
    self.statusBar.showMessage(f"Erro: {str(e)}")
else:
    # Desconectar
    self._stop_jog_and_movements()

if self.grbl:
    try:

```

```

        # Parar verificação de status
        self.grbl.poll_stop()

        # Desconectar
        self.grbl.disconnect()
        self.grbl = None
    except Exception as e:
        logger.error(f"Erro ao desconectar: {str(e)}", exc_info=True)

    self.is_connected = False
    self.machine_status = "Desconectado"
    self.connect_button.setText("Conectar")
    self.machine_state_label.setText("Desconectado")
    self.statusBar.showMessage("Desconectado")
    self.toggle_controls(False)

def _setup_callbacks(self):
    """Configura callbacks para eventos do GrblStreamer"""
    if not self.grbl:
        return

    # Configurar callbacks para o grbl-streamer
    # (A API exata pode variar com base na implementação da biblioteca)
    self.grbl.register_status_callback(self._on_status_update)
    self.grbl.register_response_callback(self._on_response_received)
    self.grbl.register_error_callback(self._on_error_message)

def _start_status_polling(self):
    """Inicia polling periódico de status"""
    if not self.is_connected or not self.grbl:
        return

    # Criar thread de consulta de status (se a lib não fizer isso automaticamente)
    self.status_thread = threading.Thread(target=self._status_polling_thread, daemon=True)
    self.status_thread.start()

def _status_polling_thread(self):
    """Thread para consulta periódica de status"""
    while self.is_connected and self.grbl:
        try:
            # Requisição de status ao GRBL a cada 200ms
            self.grbl.get_status()
            time.sleep(0.2)
        except Exception as e:
            logger.error(f"Erro na consulta de status: {str(e)}", exc_info=True)
            time.sleep(0.5) # Maior intervalo em caso de erro

def _on_status_update(self, status_data):
    """Callback quando o status é atualizado pelo grbl-streamer"""
    try:
        # Interpretar os dados de status recebidos do grbl-streamer

```

```

# (O formato exato pode variar com base na implementação da biblioteca)

# Obtém o estado da máquina
state = status_data.get('state', 'Unknown')

# Obtém a posição
position = status_data.get('position', {})
if position:
    self.current_position = {
        'x': position.get('x', 0),
        'y': position.get('y', 0),
        'z': position.get('z', 0)
    }

# Emite sinais para atualizar a interface
self.machine_status = state
self.signals.status_update.emit(self.current_position)

except Exception as e:
    logger.error(f"Erro ao processar status: {str(e)}", exc_info=True)

def _on_response_received(self, response):
    """Callback quando uma resposta é recebida pelo grbl-streamer"""
    try:
        self.last_response = response
        self.signals.response_received.emit(response)
    except Exception as e:
        logger.error(f"Erro ao processar resposta: {str(e)}", exc_info=True)

def _on_error_message(self, error):
    """Callback quando um erro é recebido pelo grbl-streamer"""
    try:
        self.last_error = error
        self.signals.error_message.emit(error)
    except Exception as e:
        logger.error(f"Erro ao processar mensagem de erro: {str(e)}", exc_info=True)

def toggle_controls(self, enabled):
    """Habilita/desabilita os controles com base no estado de conexão"""
    # Controles de eixos e botões de movimento
    self.x_minus_button.setEnabled(enabled)
    self.x_plus_button.setEnabled(enabled)
    self.y_minus_button.setEnabled(enabled)
    self.y_plus_button.setEnabled(enabled)

    # Botões direcionais
    self.dir_up_button.setEnabled(enabled)
    self.dir_down_button.setEnabled(enabled)
    self.dir_left_button.setEnabled(enabled)
    self.dir_right_button.setEnabled(enabled)
    self.jog_step.setEnabled(enabled)

```

```

# Campos de entrada
self.x_distance.setEnabled(enabled)
self.y_distance.setEnabled(enabled)
self.feed_rate.setEnabled(enabled)
self.feed_slider.setEnabled(enabled)

# Botões de controle
self.home_button.setEnabled(enabled)
self.unlock_button.setEnabled(enabled)
self.zero_button.setEnabled(enabled)
self.stop_button.setEnabled(enabled)
self.fast_mode_check.setEnabled(enabled)

# Modos de movimento
self.mode_absolute.setEnabled(enabled)
self.mode_relative.setEnabled(enabled)

# Console de comandos
self.command_input.setEnabled(enabled)
self.send_button.setEnabled(enabled)

```

```

def update_ui_state(self):
    """Atualiza o estado da interface com base no status da máquina"""
    # Atualizar texto de status
    self.machine_state_label.setText(self.machine_status)

    # Habilitar/desabilitar botão de reset com base no estado
    is_alarm = self.machine_status == "Alarm"
    self.reset_button.setEnabled(is_alarm)

    # Colorir o status da máquina e aplicar estilo ao botão de reset
    if self.machine_status == "Idle":
        self.machine_state_label.setStyleSheet("color: green;")
        self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")
    elif self.machine_status == "Run" or self.machine_status == "Jog":
        self.machine_state_label.setStyleSheet("color: blue;")
        self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")
    elif self.machine_status == "Alarm":
        self.machine_state_label.setStyleSheet("color: red;")
        self.reset_button.setStyleSheet("background-color: orange; color: white; font-weight: bold;")
    elif self.machine_status == "Hold":
        self.machine_state_label.setStyleSheet("color: orange;")
        self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")
    else:
        self.machine_state_label.setStyleSheet("")
        self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")

    # Atualizar display de posição
    self.position_display.update_position(self.current_position)

```

```

def update_feed_rate(self, value):
    """Atualiza o campo de texto quando o slider muda"""
    self.feed_rate.setText(str(value))

def update_feed_slider(self):
    """Atualiza o slider quando o campo de texto muda"""
    try:
        value = int(self.feed_rate.text())
        # Limitar ao intervalo do slider
        value = max(100, min(5000, value))
        self.feed_slider.setValue(value)
    except ValueError:
        pass

def toggle_fast_mode(self):
    """Ativa/desativa o modo rápido (desabilita verificação de porta)"""
    if not self.is_connected or not self.grbl:
        return

    try:
        if self.fast_mode_check.isChecked():
            # Desativar verificação de porta para movimentos mais rápidos
            self.grbl.send_immediately("$10=0")
            self.statusBar.showMessage("Modo rápido ativado (verificação de porta desativada)")
        else:
            # Reativar verificação de porta
            self.grbl.send_immediately("$10=1")
            self.statusBar.showMessage("Modo rápido desativado (verificação de porta ativada)")
    except Exception as e:
        logger.error(f"Erro ao alternar modo rápido: {str(e)}", exc_info=True)
        self.statusBar.showMessage(f"Erro: {str(e)}")

def set_motion_mode(self, mode):
    """Define o modo de movimento (absoluto/passo a passo ou relativo/contínuo)"""
    if not self.is_connected or not self.grbl:
        return

    if mode == "G90":
        self.mode_absolute.setChecked(True)
        self.mode_relative.setChecked(False)
        self.mode_absolute.setText("Passo a Passo (G90)")
        self.mode_relative.setText("Contínuo (G91)")
    else:
        self.mode_absolute.setChecked(False)
        self.mode_relative.setChecked(True)
        self.mode_absolute.setText("Passo a Passo (G90)")
        self.mode_relative.setText("Contínuo (G91)")

# Atualiza o modo atual
self.current_motion_mode = mode

```

```
# Envia o comando para a máquina
self.grbl.send_immediately(mode)
self.last_command = mode
self.last_command_label.setText(mode)

# Atualiza a mensagem de status com o novo modo
self.statusBar.showMessage(f"Modo de movimento alterado para {mode}")
```

```
def start_jog(self, axis, direction):
    """Inicia movimento jog para um eixo (movimento discreto)"""
    if not self.is_connected or not self.grbl:
        return

    try:
        logger.debug(f"Iniciando jog discreto: eixo={axis}, direção={direction}")

        # Determina a distância de movimento
        if axis == "X":
            distance = float(self.x_distance.text()) * direction
            command = f"G91\nG0 X{distance} F{self.feed_rate.text()}"
        else: # Y
            distance = float(self.y_distance.text()) * direction
            command = f"G91\nG0 Y{distance} F{self.feed_rate.text()}"

        # Envia o comando
        self.grbl.send_gcode(command)
        self.last_command = command
        self.last_command_label.setText(command)

    except Exception as e:
        logger.error(f"Erro ao iniciar jog: {str(e)}", exc_info=True)
        self.statusBar.showMessage(f"Erro ao iniciar movimento: {str(e)}")

def stop_jog(self):
    """Para o movimento jog quando o botão é liberado"""
    # Para movimento discreto, não precisamos fazer nada especial
    pass

def _continue_jog_movement(self, axis, direction, distance, feed_rate):
    """Continua o movimento jog enviando comandos incrementais pequenos"""
    if not self.is_connected or not self.grbl or not self.jogging:
        if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
            self.jog_timer.stop()
        return

    # Cria o comando para continuar o movimento
    if axis.upper() == 'X':
        jog_command = f"G00 X{distance} F{feed_rate}"
    else: # Y
        jog_command = f"G00 Y{distance} F{feed_rate}"
```

```

# Envia o comando usando o método disponível
try:
    self.grbl.send_immediately(jog_command)
except Exception as e:
    logger.error(f"Erro ao continuar jog: {str(e)}", exc_info=True)
    if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
        self.jog_timer.stop()

def _continue_jog(self):
    """Envia comandos contínuos de movimento enquanto o botão está pressionado"""
    if not self.is_connected or not self.grbl:
        return

    try:
        # Cria um novo comando usando G1 (movimento linear) em vez de G0
        # G1 é mais suave para movimentos contínuos
        if self.jog_axis.upper() == 'X':
            command = f"G1 X{self.jog_direction * self.jog_step_size} F{self.jog_feed_rate}"
        else: # Y
            command = f"G1 Y{self.jog_direction * self.jog_step_size} F{self.jog_feed_rate}"

        # Envia o comando sem interromper fluxos anteriores
        self.grbl.send_immediately(command)

    except Exception as e:
        logger.error(f"Erro ao continuar jog: {str(e)}", exc_info=True)
        self.jog_timer.stop()

def start_continuous_jog(self, axis, direction):
    """
    Inicia movimento passo a passo (G90) ou contínuo (G91), dependendo do modo atual.
    """
    if not self.is_connected or not self.grbl:
        return

    # Para qualquer jog anterior antes de iniciar o novo
    self.stop_continuous_jog()
    time.sleep(0.05) # Pequena pausa para garantir que o GRBL processe o comando de parada

    try:
        # Obtém o modo atual (G90 = Absoluto, G91 = Relativo)
        is_absolute_mode = (self.current_motion_mode == "G90")

        logger.debug(f"Iniciando jog: eixo={axis}, direção={direction}, modo={f'G90 (Absoluto/Passo a Passo)' if is_absolute_mode else 'G91 (Relativo)'}")

        # Obtém a velocidade e o tamanho do passo
        try:
            feed_rate = int(self.feed_rate.text())
            feed_rate = max(100, min(feed_rate, 5000)) # Limita a velocidade para segurança
        except ValueError:
            feed_rate = 500 # Valor seguro padrão
    
```

```

try:
    step_size = float(self.jog_step.text())
    step_size = max(0.1, min(step_size, 10)) # Limita o tamanho do passo entre 0.1 e 10mm
except ValueError:
    step_size = 1.0 # Valor padrão seguro

# Movimento com base no modo atual
if is_absolute_mode:
    # G90 MODO ABSOLUTO/PASSO A PASSO: Movimento incremental relativo à posição atual
    # CORREÇÃO: Primeiro garantimos que estamos em modo relativo para o movimento
    self.grbl.send_immediately("G91")

    # Calcula a distância de movimento baseada no passo configurado
    distance = step_size * direction

    # Cria um comando de movimento incremental apesar de estar em modo G90 na interface
    if axis.upper() == 'X':
        jog_command = f"G0 X{distance} F{feed_rate}"
    else: # Y
        jog_command = f"G0 Y{distance} F{feed_rate}"

    # Envia o comando de movimento incremental
    self.grbl.send_immediately(jog_command)
    self.last_command = jog_command
    self.last_command_label.setText(jog_command)

    # Restaura o modo absoluto ao final do movimento
    QTimer.singleShot(100, lambda: self.grbl.send_immediately("G90"))

    # Não define estado de jogging, pois é uma operação única
    self.jogging = False

else:
    # G91 MODO RELATIVO/CONTÍNUO: Movimento contínuo enquanto o botão é pressionado
    # Usar o comando $J= para jog contínuo
    distance_continuous = 50 * direction # Distância grande o suficiente para movimento contínuo

    if axis.upper() == 'X':
        jog_command = f"$J=G91 X{distance_continuous} F{feed_rate}"
    else: # Y
        jog_command = f"$J=G91 Y{distance_continuous} F{feed_rate}"

    # Imediatamente envia o comando de jog
    self.grbl.send_immediately(jog_command)
    self.last_command = jog_command
    self.last_command_label.setText(jog_command)

    # Define flags de controle apenas para modo contínuo
    self.jogging = True
    self.current_jog_axis = axis

```

```
self.current_jog_direction = direction
```

```
except Exception as e:
```

```
    logger.error(f'Erro ao iniciar jog: {str(e)}", exc_info=True)
```

```
    self.statusBar.showMessage(f'Erro ao iniciar movimento: {str(e)}")
```

```
def stop_continuous_jog(self):
```

```
    """
```

```
    Para o movimento jog contínuo quando o botão é liberado.
```

```
    No modo G90, esta função não faz nada pois o movimento é único.
```

```
    """
```

```
    if not self.is_connected or not self.grbl or not self.jogging:
```

```
        return
```

```
    logger.debug("Parando jog contínuo")
```

```
    try:
```

```
        # Comando de feed hold para parar imediatamente
```

```
        self.grbl.send_immediately("!")
```

```
        # Imediatamente após, envia comando de ciclo contínuo para liberar o hold
```

```
        QTimer.singleShot(50, lambda: self.grbl.send_immediately("~"))
```

```
        # Limpa o estado de jog
```

```
        self.jogging = False
```

```
        self.current_jog_axis = None
```

```
        self.current_jog_direction = None
```

```
except Exception as e:
```

```
    logger.error(f'Erro ao parar jog contínuo: {str(e)}", exc_info=True)
```

```
def _stop_jog_and_movements(self):
```

```
    """Para todos os movimentos ativos"""
```

```
    if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
```

```
        self.jog_timer.stop()
```

```
    if not self.is_connected or not self.grbl:
```

```
        return
```

```
    try:
```

```
        # Envia comando de parada
```

```
        self.grbl.send_immediately("!")
```

```
except Exception as e:
```

```
    logger.error(f'Erro ao parar movimentos: {str(e)}", exc_info=True)
```

```
def _send_resume_after_stop(self):
```

```
    """Envia comando de retomada após parada"""
```

```
    if not self.is_connected or not self.grbl:
```

```
        return
```

try:

```
# Enviar comando de retomada usando o método correto
self.grbl.send_realtime("~")
```

except Exception as e:

```
logger.error(f'Erro ao enviar comando de retomada: {str(e)}', exc_info=True)
```

def home(self):

```
"""Envia comando para ir para home"""
```

```
if not self.is_connected or not self.grbl:
```

```
    return
```

try:

```
# Usar send_immediately
```

```
self.grbl.send_immediately("$H")
```

```
self.last_command = "$H"
```

```
self.last_command_label.setText("$H (Home)")
```

except Exception as e:

```
logger.error(f'Erro ao enviar comando home: {str(e)}', exc_info=True)
```

def unlock(self):

```
"""Desbloqueia a máquina após um alarme"""
```

```
if not self.is_connected or not self.grbl:
```

```
    return
```

try:

```
# Usar send_immediately
```

```
self.grbl.send_immediately("$X")
```

```
self.last_command = "$X"
```

```
self.last_command_label.setText("$X (Unlock)")
```

except Exception as e:

```
logger.error(f'Erro ao enviar comando de desbloqueio: {str(e)}', exc_info=True)
```

def set_zero(self):

```
"""Define a posição atual como zero para todos os eixos"""
```

```
if not self.is_connected or not self.grbl:
```

```
    return
```

try:

```
# Usar send_immediately
```

```
self.grbl.send_immediately("G92 X0 Y0")
```

```
self.last_command = "G92 X0 Y0"
```

```
self.last_command_label.setText("G92 X0 Y0 (Set Zero)")
```

except Exception as e:

```
logger.error(f'Erro ao definir posição zero: {str(e)}', exc_info=True)
```

def emergency_stop(self):

```
"""Para todos os movimentos imediatamente e coloca a máquina em estado seguro"""
```

```
if not self.is_connected or not self.grbl:
```

```
    return
```

try:

```
logger.debug("Executando parada de emergência")
```

```
# Para qualquer timer ativo
```

```
if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
```

```
    self.jog_timer.stop()
```

```
# Sequência robusta de parada
```

```
# 1. Feed hold imediato (!)
```

```
self.grbl.send_immediately("!")
```

```
# 2. Soft reset para garantir parada completa
```

```
QTimer.singleShot(100, lambda: self.grbl.send_immediately("\x18")) # Ctrl+X
```

```
# Reset completo de estado
```

```
self.jogging = False
```

```
self.current_jog_axis = None
```

```
self.current_jog_direction = None
```

```
self.last_command = "! (Parada de Emergência)"
```

```
self.last_command_label.setText("! (Parada de Emergência)")
```

```
self.statusBar.showMessage("PARADA DE EMERGÊNCIA acionada! Use o botão 'Reset Alarme' para continuar.")
```

```
# Destaca o botão de reset para orientar o usuário
```

```
self.reset_button.setStyleSheet("background-color: orange; color: white; font-weight: bold; font-size: 14px; border: 2px solid orange;")
```

except Exception as e:

```
logger.error(f"Erro na parada de emergência: {str(e)}", exc_info=True)
```

```
self.statusBar.showMessage(f"Erro: {str(e)}")
```

def send_command(self):

```
    """Envia um comando G-code personalizado"""
```

```
    if not self.is_connected or not self.grbl:
```

```
        return
```

```
    command = self.command_input.text()
```

```
    if not command:
```

```
        return
```

try:

```
# Usar send_immediately em vez de send_gcode
```

```
self.grbl.send_immediately(command)
```

```
self.last_command = command
```

```
self.last_command_label.setText(command)
```

```
self.command_input.clear()
```

except Exception as e:

```
logger.error(f"Erro ao enviar comando: {str(e)}", exc_info=True)
```

```
self.statusBar.showMessage(f"Erro ao enviar comando: {str(e)}")
```

def on_response_received(self, response):

```
    """Manipula a resposta recebida"""
```

```

self.last_response_label.setText(response)
self.comm_log.setText(response)

def on_status_update(self, position):
    """Atualiza a exibição de posição na interface"""
    # Esta função é chamada pelo sinal emitido no _on_status_update
    self.position_display.update_position(position)

def on_error_message(self, error):
    """Exibe mensagem de erro na barra de status"""
    self.statusBar.showMessage(error)

def closeEvent(self, event):
    """Manipula o evento de fechamento da janela"""
    # Para todos os movimentos e desconecta
    if self.is_connected and self.grbl:
        self._stop_jog_and_movements()
        try:
            self.grbl.close()
        except Exception:
            pass

    event.accept()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = GRBLController()
    window.show()
    sys.exit(app.exec())

```

File: aoi_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```

import time
from .cnc_controller import GRBLCNCController
from .camera_controller import CameraController
from .position_manager import InspectionPositionManager, InspectionPosition, InspectionSequence

class CNCAOIController:
    """
    Controlador principal que integra o movimento CNC e captura de imagem
    para aplicações de Inspeção Óptica Automatizada (AOI).
    """

    def __init__(self, serial_port=None, camera_interface=None):
        """
        Inicializa o controlador AOI.

        Args:
            serial_port: Porta serial para conexão com a CNC (opcional)

```

```

        camera_interface: Interface para a câmera (opcional)
        """

        self.cnc = GRBLCNCController()
        self.camera = CameraController(camera_interface)
        self.position_manager = InspectionPositionManager()
        self.is_running_sequence = False
        self.current_sequence = None
        self.current_position_index = -1

def connect_cnc(self, port=None, baudrate=115200):
    """Conecta à máquina CNC."""
    return self.cnc.connect(port, baudrate)

def run_gcode_file(self, filename, callback=None):
    """
    Executa um arquivo G-CODE diretamente

    Args:
        filename: Caminho do arquivo G-CODE
        callback: Função chamada após cada captura
    """

    from aoi_lib.gcode_manager import GCodeManager
    gcode_manager = GCodeManager()

    # Carrega o G-CODE para uma sequência temporária
    gcode_data = gcode_manager.read_gcode_file(filename)

    if not gcode_data:
        raise ValueError(f"Erro ao carregar arquivo G-CODE: {filename}")

    # Cria posições a partir dos dados do G-CODE
    positions = []
    for pos_data in gcode_data['positions']:
        position = InspectionPosition(
            pos_data['name'],
            pos_data['x'],
            pos_data['y'],
            pos_data['camera_params']
        )
        positions.append(position)

    # Cria sequência temporária
    temp_sequence = InspectionSequence(gcode_data['sequence_name'])
    for pos in positions:
        temp_sequence.add_position(pos)

    # Executa a sequência
    return self.run_sequence(temp_sequence.name, callback)

def disconnect_cnc(self):
    """Desconecta da máquina CNC."""

```

```
if self.cnc.is_connected:
    return self.cnc.disconnect()
return True
```

```
def connect_camera(self, camera_id=0):
    """Conecta à câmera."""
    return self.camera.connect(camera_id)
```

```
def add_inspection_position(self, name, x, y, camera_params=None):
    """
    Adiciona uma posição de inspeção ao gerenciador.
```

Args:

name: Nome descritivo da posição
x, y: Coordenadas
camera_params: Parâmetros específicos da câmera (opcional)

```
    """
    position = InspectionPosition(name, x, y, camera_params)
    self.position_manager.add_position(position)
    return position
```

```
def add_current_position(self, name, camera_params=None):
    """Adiciona a posição atual como uma posição de inspeção."""
    position = self.cnc.get_current_position()
    return self.add_inspection_position(name, position['x'], position['y'], camera_params)
```

```
def create_sequence(self, name, positions=None):
    """Cria uma nova sequência de inspeção."""
    sequence = InspectionSequence(name)
    if positions:
        for pos in positions:
            sequence.add_position(pos)
    self.position_manager.add_sequence(sequence)
    return sequence
```

```
def run_sequence(self, sequence_name, callback=None):
    """
    Executa uma sequência de inspeção completa.
```

Args:

sequence_name: Nome da sequência a executar
callback: Função chamada após cada captura com imagem e metadados

```
    """
    sequence = self.position_manager.get_sequence(sequence_name)
    if not sequence:
        raise ValueError(f"Sequência '{sequence_name}' não encontrada")
```

```
    self.is_running_sequence = True
    self.current_sequence = sequence
    self.current_position_index = 0
```

```

# Iniciar execução da sequência
return self._process_next_position(callback)

def _process_next_position(self, callback=None):
    """Processa a próxima posição na sequência."""
    if not self.is_running_sequence or self.current_position_index >= len(self.current_sequence.positions):
        self.is_running_sequence = False
        return None

    # Obtém a próxima posição
    position = self.current_sequence.positions[self.current_position_index]

    # Move para a posição
    self.cnc.move_to_absolute_position(position.x, position.y)

    # Verifica se chegou na posição
    while self.cnc.is_moving():
        time.sleep(0.1)

    # Captura a imagem
    image = self.camera.capture(position.camera_params)

    # Prepara dados para callback
    result = {
        "position": position,
        "image": image,
        "timestamp": time.time(),
        "sequence": self.current_sequence.name,
        "index": self.current_position_index
    }

    # Avança para a próxima posição
    self.current_position_index += 1

    # Chama o callback se fornecido
    if callback:
        callback(result)

    return result

def stop_sequence(self):
    """Para a execução da sequência atual."""
    self.is_running_sequence = False

def save_positions(self, filename):
    """Salva todas as posições e sequências em um arquivo."""
    return self.position_manager.save_to_file(filename)

def load_positions(self, filename):
    """Carrega posições e sequências de um arquivo."""
    return self.position_manager.load_from_file(filename)

```

File: camera_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
class CameraController:
```

```
    """
```

```
    Classe para controlar uma câmera e capturar imagens.
```

```
    Pode ser adaptada para diferentes interfaces de câmera.
```

```
    """
```

```
    def __init__(self, camera_interface=None):
```

```
        """
```

```
        Inicializa o controlador de câmera.
```

```
        Args:
```

```
            camera_interface: Interface de câmera personalizada (opcional)
```

```
        """
```

```
        self.camera = camera_interface
```

```
        self.is_connected = False
```

```
        self.last_error = ""
```

```
    def connect(self, camera_id=0):
```

```
        """
```

```
        Conecta à câmera.
```

```
        Args:
```

```
            camera_id: ID da câmera (para OpenCV)
```

```
        Returns:
```

```
            bool: True se conectado com sucesso
```

```
        """
```

```
        if self.camera:
```

```
            # Usa a interface fornecida
```

```
            try:
```

```
                self.is_connected = self.camera.connect()
```

```
                return self.is_connected
```

```
            except Exception as e:
```

```
                self.last_error = str(e)
```

```
                return False
```

```
        else:
```

```
            # Implementação padrão com OpenCV
```

```
            try:
```

```
                import cv2
```

```
                self.camera = cv2.VideoCapture(camera_id)
```

```
                self.is_connected = self.camera.isOpened()
```

```
                return self.is_connected
```

```
            except Exception as e:
```

```
                self.last_error = str(e)
```

```
                return False
```

```
    def disconnect(self):
```

```
        """Desconecta da câmera."""
```

```
if not self.is_connected:  
    return
```

```
try:  
    if hasattr(self.camera, 'release'):  
        self.camera.release()  
    elif hasattr(self.camera, 'disconnect'):  
        self.camera.disconnect()
```

```
    self.is_connected = False
```

```
except Exception as e:  
    self.last_error = str(e)
```

```
def trigger_capture(self, params=None):  
    """
```

Aciona o trigger da câmera para captura.

Args:

params: Parâmetros específicos da câmera para o trigger

Returns:

A imagem capturada ou None em caso de erro

```
    """
```

```
# Primeiro realiza o trigger
```

```
if not self._activate_trigger(params):  
    return None
```

```
# Em seguida, captura a imagem
```

```
return self.capture(params)
```

```
def _activate_trigger(self, params=None):  
    """
```

Ativa o hardware trigger da câmera, se disponível.

Args:

params: Parâmetros específicos para o trigger

Returns:

True se o trigger foi bem-sucedido, False caso contrário

```
    """
```

```
if not self.is_connected:  
    self.last_error = "Câmera não conectada"  
    return False
```

```
try:
```

```
# Verifica se a interface da câmera tem um método de trigger específico
```

```
if hasattr(self.camera, 'trigger'):
```

```
    # Interface personalizada com método de trigger
```

```
    return self.camera.trigger(params)
```

```

# Para câmeras OpenCV padrão, criar um atraso simulando trigger por software
# e garantir que o frame seja descartado/limpo antes da captura real
import time
if hasattr(self.camera, 'grab'):
    # Limpa o buffer de frames
    self.camera.grab()
    time.sleep(0.1) # Pequeno atraso para simular trigger
    return True

return True # Se não houver trigger específico, continua com a captura normal

except Exception as e:
    self.last_error = f"Erro ao ativar trigger: {str(e)}"
    return False

def has_hardware_trigger(self):
    """
    Verifica se a câmera possui suporte a hardware trigger.

    Returns:
        True se suporta trigger por hardware, False caso contrário
    """
    if not self.is_connected:
        return False

    # Verifica se a interface da câmera tem capacidade explícita de trigger
    return hasattr(self.camera, 'trigger') or hasattr(self.camera, 'has_hardware_trigger')

def capture(self, params=None):
    """
    Captura uma imagem da câmera.

    Args:
        params: Parâmetros específicos da câmera (exposição, etc.)

    Returns:
        A imagem capturada ou None em caso de erro
    """
    if not self.is_connected:
        self.last_error = "Câmera não conectada"
        return None

    try:
        # Para interface personalizada
        if hasattr(self.camera, 'capture'):
            image = self.camera.capture(params)
            if image is None:
                self.last_error = "A captura de imagem retornou None"
                return None
            return image

```

```

# Para OpenCV
import cv2
ret, frame = self.camera.read()
if ret and frame is not None:
    # Aplicar parâmetros se fornecidos
    if params:
        # Exemplo: ajustar brilho/contraste
        if 'brightness' in params:
            frame = cv2.convertScaleAbs(frame, alpha=params['brightness'])

    # Verifica se a imagem possui dados válidos
    if frame.size == 0 or frame.shape[0] == 0 or frame.shape[1] == 0:
        self.last_error = "Imagem capturada tem dimensões inválidas"
        return None

    return frame
else:
    self.last_error = "Falha ao capturar imagem (sem dados ou retorno negativo)"
    return None

except Exception as e:
    self.last_error = f"Erro na captura de imagem: {str(e)}"
    return None

def set_parameters(self, **kwargs):
    """
    Define parâmetros da câmera.

    Args:
        **kwargs: Parâmetros específicos da câmera
    """
    if not self.is_connected:
        return False

    try:
        # Para interface personalizada
        if hasattr(self.camera, 'set_parameters'):
            return self.camera.set_parameters(**kwargs)

        # Para OpenCV
        import cv2
        for key, value in kwargs.items():
            if key.lower() == 'exposure':
                self.camera.set(cv2.CAP_PROP_EXPOSURE, value)
            elif key.lower() == 'focus':
                self.camera.set(cv2.CAP_PROP_FOCUS, value)
            # Adicione outros parâmetros conforme necessário

        return True

    except Exception as e:

```

```
self.last_error = str(e)
return False
```

File: cnc_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
import serial
import serial.tools.list_ports
import threading
from grbl_streamer import GrblStreamer # Importa a biblioteca grbl-streamer

import logging
# Se ainda não existir, defina um logger para esta classe:
logger = logging.getLogger("GRBLCNCController")
logger.setLevel(logging.DEBUG)

class EventEmitter:
    """
    Sistema de eventos para comunicação assíncrona.
    Permite registrar múltiplos callbacks para diferentes eventos.
    """
    def __init__(self):
        self.callbacks = {}
        self.default_callbacks = []

    def connect(self, callback, event_name=None):
        """
        Conecta um callback a um evento específico ou como callback padrão.

        Args:
            callback: Função a ser chamada quando o evento ocorrer
            event_name: Nome do evento (se None, será adicionado aos callbacks padrão)
        """
        if event_name is None:
            self.default_callbacks.append(callback)
        else:
            if event_name not in self.callbacks:
                self.callbacks[event_name] = []
            self.callbacks[event_name].append(callback)

        # Retorna uma função que pode ser usada para desconectar este callback
        return lambda: self.disconnect(callback, event_name)

    def disconnect(self, callback, event_name=None):
        """
        Desconecta um callback específico.

        Args:
            callback: Callback a ser removido
        """
```

```

    event_name: Nome do evento do qual remover (se None, remove dos callbacks padrão)
    """

    if event_name is None:
        if callback in self.default_callbacks:
            self.default_callbacks.remove(callback)
        else:
            if event_name in self.callbacks and callback in self.callbacks[event_name]:
                self.callbacks[event_name].remove(callback)

def disconnect_all(self, event_name=None):
    """
    Desconecta todos os callbacks de um evento específico ou todos os eventos.

    Args:
        event_name: Nome do evento para limpar (se None, limpa todos os eventos)
    """
    if event_name is None:
        self.callbacks.clear()
        self.default_callbacks.clear()
    else:
        if event_name in self.callbacks:
            self.callbacks[event_name].clear()

def emit(self, *args, event_name=None, **kwargs):
    """
    Emite um evento, chamando todos os callbacks registrados.

    Args:
        *args: Argumentos posicionais para passar aos callbacks
        event_name: Nome do evento a ser emitido (se None, apenas callbacks padrão são chamados)
        **kwargs: Argumentos nomeados para passar aos callbacks
    """
    # Chama todos os callbacks padrão
    for callback in self.default_callbacks:
        try:
            callback(*args, **kwargs)
        except Exception as e:
            print(f"Erro em callback para evento: {e}")

    # Se um nome de evento for especificado, chama esses callbacks também
    if event_name is not None and event_name in self.callbacks:
        for callback in self.callbacks[event_name]:
            try:
                callback(*args, **kwargs)
            except Exception as e:
                print(f"Erro em callback para evento '{event_name}': {e}")

def has_listeners(self, event_name=None):
    """
    Verifica se há callbacks registrados para um evento específico ou para qualquer evento.

```

Args:

event_name: Nome do evento a verificar (se None, verifica todos)

Returns:

bool: True se houver callbacks registrados, False caso contrário

"""

if event_name is None:

return bool(self.default_callbacks) or any(bool(callbacks) for callbacks in self.callbacks.values())

else:

return event_name in self.callbacks and bool(self.callbacks[event_name])

class GRBLCNCController:

"""

Classe para controlar uma máquina CNC usando GRBL.

Implementada usando a biblioteca grbl-streamer.

"""

def __init__(self):

"""Inicializa o controlador CNC."""

self.grbl = None

self.lock = threading.Lock()

self.is_connected = False

self.current_position = {'x': 0, 'y': 0, 'z': 0}

self.machine_status = "Disconnected"

self.last_response = ""

self.last_error = ""

Eventos para comunicação assíncrona

self.response_received = EventEmitter()

self.status_update = EventEmitter()

self.position_update = EventEmitter()

self.error_message = EventEmitter()

Thread para consulta periódica de status

self.status_thread = None

self.running = False

Controle de jog contínuo

self.jogging = False

self.current_jog_command = None

self.steps_to_mm_factor = 1.0

def connect(self, port=None, baudrate=115200):

"""

Conecta à máquina CNC.

Args:

port: Porta serial (None para autodetecção)

baudrate: Taxa de transmissão

Returns:

```

        bool: True se conectado com sucesso
        """
    try:
        # Autodetectar porta se não especificada
        if port is None:
            ports = [p.device for p in serial.tools.list_ports.comports()]
            if not ports:
                self.last_error = "Nenhuma porta serial encontrada"
                return False
            port = ports[0]

        # Inicializa e configura o GrblStreamer
        self.grbl = GrblStreamer(port=port, baud=baudrate)

        # Configura callbacks para eventos do GrblStreamer
        self._setup_callbacks()

        # Desbloqueia a máquina
        self.send_command("$X", priority=True)

        # Inicia thread de consulta de status
        self.running = True
        self.status_thread = threading.Thread(target=self._status_polling_thread, daemon=True)
        self.status_thread.start()

        self.is_connected = True
        self.machine_status = "Idle" # Estado inicial presumido
        return True

    except Exception as e:
        self.last_error = str(e)
        return False

def _setup_callbacks(self):
    """
    Configura callbacks para eventos do GrblStreamer.
    Esta função deve ser adaptada com base na API real da biblioteca.
    """
    # Configuração para receber atualizações de status da máquina
    self.grbl.register_status_callback(self._on_status_update)

    # Configuração para receber respostas de comandos
    self.grbl.register_response_callback(self._on_response_received)

    # Configuração para receber notificações de erro
    self.grbl.register_error_callback(self._on_error_message)

def _status_polling_thread(self):
    """Thread para consulta periódica de status."""
    while self.running and self.is_connected:
        try:

```

```

        # Requisição de status ao GRBL a cada 200ms
        if self.grbl:
            self.grbl.get_status()
            time.sleep(0.2)
        except Exception:
            time.sleep(0.5) # Maior intervalo em caso de erro

```

```

def disconnect(self):
    """Desconecta da máquina CNC."""
    # Garante que o jog é parado antes de desconectar
    if self.jogging:
        self.stop_continuous_jog()

    self.running = False
    if self.status_thread:
        self.status_thread.join(1.0) # Espera até 1 segundo pela thread

    if self.grbl:
        try:
            self.grbl.close()
            self.grbl = None
        except Exception as e:
            self.last_error = str(e)

    self.is_connected = False
    self.machine_status = "Disconnected"

```

```

def is_moving(self):
    """Verifica se a máquina está em movimento."""
    return self.machine_status == "Run" or self.machine_status == "Jog"

```

```

def wait_for_idle(self, timeout=30):
    """
    Aguarda até que a máquina esteja ociosa.

```

Args:

timeout: Tempo máximo de espera em segundos

Returns:

bool: True se ficou ocioso, False se atingiu timeout

```

    """
    start_time = time.time()
    while self.is_moving():
        if time.time() - start_time > timeout:
            return False
        time.sleep(0.1)
    return True

```

```

def get_current_position(self):
    """
    Retorna a posição atual do CNC.

```

Certifica-se de retornar uma cópia do dicionário para evitar alteração acidental.

"""

```
logger.debug(f"get_current_position: Retornando posição atual: {self.current_position if hasattr(self, 'current_position')
if hasattr(self, 'current_position'):
    return self.current_position.copy()
return {'x': 0, 'y': 0, 'z': 0}
```

```
def move_to_absolute_position(self, x=None, y=None, feed_rate=1000):
```

"""

Move para uma posição absoluta.

Args:

x, y: Coordenadas de destino (None para não mover esse eixo)

feed_rate: Velocidade de avanço em mm/min

"""

```
if not self.is_connected:
```

```
    return False
```

```
# Muda para modo absoluto
```

```
self.send_command("G90", priority=True)
```

```
# Constrói o comando
```

```
command = "G1"
```

```
if x is not None:
```

```
    command += f" X{x}"
```

```
if y is not None:
```

```
    command += f" Y{y}"
```

```
command += f" F{feed_rate}"
```

```
# Envia o comando
```

```
self.send_command(command)
```

```
return True
```

```
def move_relative(self, x=0, y=0, feed_rate=1000):
```

"""

Move em relação à posição atual.

Args:

x, y: Distância a mover em cada eixo

feed_rate: Velocidade de avanço em mm/min

"""

```
if not self.is_connected:
```

```
    return False
```

```
# Muda para modo relativo
```

```
self.send_command("G91", priority=True)
```

```
# Constrói o comando
```

```
command = f"G1 X{x} Y{y} F{feed_rate}"
```

```
# Envia o comando
```

```
self.send_command(command)
return True
```

```
def start_continuous_jog(self, axis, direction, feed_rate=1000):
```

```
    """
```

Inicia um movimento jog contínuo em um eixo específico.

O movimento continua até que stop_continuous_jog seja chamado.

Args:

axis: Eixo a mover ('X' ou 'Y')

direction: Direção do movimento (1 ou -1)

feed_rate: Velocidade em mm/min

Returns:

bool: True se o comando foi iniciado com sucesso

```
    """
```

```
if not self.is_connected or self.jogging:
```

```
    return False
```

```
# Para qualquer movimento anterior e garante modo relativo
```

```
self.emergency_stop()
```

```
time.sleep(0.1) # Pequena pausa para garantir que o GRBL processou o comando de parada
```

```
self.send_command("~", priority=True) # Retoma após parada
```

```
self.send_command("G91", priority=True) # Modo relativo
```

```
# Em GRBL 1.1, podemos usar o comando de jog para movimento contínuo
```

```
# A sintaxe é: $J=G91 X[dist] Y[dist] F[feed]
```

```
distance = 1000 * direction # Distância grande para simular movimento contínuo
```

```
if axis.upper() == 'X':
```

```
    jog_command = f"$J=G91 X{distance} F{feed_rate}"
```

```
else: # assume Y
```

```
    jog_command = f"$J=G91 Y{distance} F{feed_rate}"
```

```
self.jogging = True
```

```
self.current_jog_command = jog_command
```

```
# Envia comando de jog com máxima prioridade
```

```
success = self.send_command(jog_command, priority=True)
```

```
return success
```

```
def stop_continuous_jog(self):
```

```
    """
```

Para o movimento jog contínuo atual.

Returns:

bool: True se o comando foi enviado com sucesso

```
    """
```

```
if not self.is_connected or not self.jogging:
```

```
    return False
```

```
# O comando para cancelar jog no GRBL é o feed cancel (!)
```

```
success = self.send_command("!", priority=True)
```

```
# Após um breve intervalo, envia comando de retomada para liberar a máquina
```

```
time.sleep(0.1)
```

```
self.send_command("~", priority=True)
```

```
self.jogging = False
```

```
self.current_jog_command = None
```

```
return success
```

```
def home(self):
```

```
    """Envia a máquina para a posição home."""
```

```
    if not self.is_connected:
```

```
        return False
```

```
    self.send_command("$H")
```

```
    return True
```

```
def set_zero(self):
```

```
    """Define a posição atual como zero."""
```

```
    if not self.is_connected:
```

```
        return False
```

```
    self.send_command("G92 X0 Y0")
```

```
    return True
```

```
def unlock(self):
```

```
    """Desbloqueia a máquina."""
```

```
    if not self.is_connected:
```

```
        return False
```

```
    self.send_command("$X", priority=True)
```

```
    return True
```

```
def emergency_stop(self):
```

```
    """Para todos os movimentos imediatamente."""
```

```
    if not self.is_connected:
```

```
        return False
```

```
    self.grbl.send_realtime_command("!")
```

```
    self.jogging = False # Atualiza o estado de jog
```

```
    return True
```

```
def send_command(self, command, priority=False):
```

```
    """
```

```
    Envia um comando personalizado ao GRBL.
```

```
    Args:
```

```
command: Comando G-code
priority: True para enviar com prioridade
```

```
"""
```

```
if not self.is_connected or not self.grbl:
    return False
```

```
try:
```

```
    # Comandos de controle realtime (!, ~, ?) são tratados especialmente
```

```
    if command in ["!", "~", "?"]:
```

```
        self.grbl.send_realtime_command(command)
```

```
    # Comandos prioritários são enviados com prioridade
```

```
    elif priority:
```

```
        self.grbl.send_priority_command(command)
```

```
    # Comandos normais
```

```
    else:
```

```
        self.grbl.send_gcode(command)
```

```
    return True
```

```
except Exception as e:
```

```
    self.last_error = str(e)
```

```
    self.error_message.emit(str(e))
```

```
    return False
```

```
def _on_response_received(self, response):
```

```
    """Callback quando uma resposta é recebida."""
```

```
    self.last_response = response
```

```
    self.response_received.emit(response)
```

```
def _on_status_update(self, status_data):
```

```
    """Callback quando o status da máquina é atualizado."""
```

```
    logger.debug(">> _on_status_update chamado com status_data: %s", status_data)
```

```
    # Extraí o estado da máquina
```

```
    state = status_data.get('state', 'Unknown')
```

```
    self.machine_status = state
```

```
    logger.debug("Estado da máquina extraído: %s", state)
```

```
    # Extraí a posição (espera-se um dicionário com chaves 'x', 'y' e 'z')
```

```
    position = status_data.get('position', None)
```

```
    if position:
```

```
        new_position = {
```

```
            'x': position.get('x', 0),
```

```
            'y': position.get('y', 0),
```

```
            'z': position.get('z', 0)
```

```
        }
```

```
        logger.debug("Posição extraída: %s", new_position)
```

```
        if new_position['x'] == 0 and new_position['y'] == 0:
```

```
            logger.warning("As coordenadas X e Y continuam 0 mesmo após movimento. status_data recebido: %s", status_data)
```

```
            self.current_position = new_position
```

```
            self.position_update.emit(self.current_position)
```

```
    else:
```

```
        logger.warning("status_data não contém a chave 'position': %s", status_data)
```

```

status_string = f"<{state}|MPos:{self.current_position['x']:.3f},{self.current_position['y']:.3f},{self.current_position['z']:.3f}>"
logger.debug("String de status formada: %s", status_string)
self.status_update.emit(status_string)

def _on_error_message(self, error):
    """Callback quando ocorre um erro."""
    self.last_error = error
    self.error_message.emit(error)

```

File: gcode_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```

"""

```

Módulo para gerenciamento de G-CODE na aplicação de inspeção AOI.
Permite criar, ler e manipular arquivos G-CODE para controle de percursos de inspeção.

```

"""

```

```

import os
import re
import time
import logging
from datetime import datetime

logger = logging.getLogger("GCodeManager")

```

```

class GCodeManager:

```

```

    """

```

Gerencia operações de G-CODE para o sistema AOI, incluindo:

- Geração de G-CODE a partir de posições de inspeção
- Leitura de arquivos G-CODE para criar sequências de inspeção
- Inclusão de comandos especiais para captura de imagens

```

    """

```

Definição de comandos especiais como comentários em G-CODE

CAPTURE_IMAGE_CMD = ";AOI_CAPTURE_IMAGE"

POSITION_NAME_CMD = ";AOI_POSITION_NAME:"

SEQUENCE_NAME_CMD = ";AOI_SEQUENCE_NAME:"

CAMERA_PARAMS_CMD = ";AOI_CAMERA_PARAMS:"

```

def __init__(self):

```

```

    """Inicializa o gerenciador de G-CODE"""

```

```

    self.feed_rate = 1000 # Taxa de avanço padrão (mm/min)

```

```

    self.safe_z_height = 5 # Altura Z para movimentos seguros

```

```

    self.use_z_axis = False # Se verdadeiro, inclui movimentos no eixo Z

```

```

def generate_gcode_from_sequence(self, sequence, include_header=True):

```

```

    """

```

Gera G-CODE a partir de uma sequência de inspeção

Args:

sequence: Objeto InspectionSequence contendo as posições

include_header: Se True, inclui cabeçalho com informações da sequência

Returns:

Uma string contendo o G-CODE completo

"""

gcode_lines = []

Adiciona cabeçalho com informações

if include_header:

```
gcode_lines.extend([
    f"; AOI Inspection Sequence Generated G-CODE",
    f"; Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
    f"{self.SEQUENCE_NAME_CMD} {sequence.name}",
    f"; Total positions: {len(sequence.positions)}",
    f"; Feed rate: {self.feed_rate} mm/min",
    ";",
])
```

Configurações iniciais

```
gcode_lines.extend([
    "G21 ; Set units to mm",
    "G90 ; Set absolute positioning",
    f"F{self.feed_rate} ; Set feed rate"
])
```

Para cada posição na sequência

for i, position in enumerate(sequence.positions):

Adiciona comentário com o nome da posição

```
gcode_lines.append(f"\n; Position {i+1}: {position.name}")
```

```
gcode_lines.append(f"{self.POSITION_NAME_CMD} {position.name}")
```

Adiciona parâmetros da câmera se existirem

if position.camera_params:

```
params_str = ",".join([f"{k}:{v}" for k, v in position.camera_params.items()])
```

```
gcode_lines.append(f"{self.CAMERA_PARAMS_CMD} {params_str}")
```

Adiciona movimento para a posição

```
gcode_lines.append(f"G0 X{position.x:.3f} Y{position.y:.3f} ; Move to position")
```

Adiciona comando para captura de imagem

```
gcode_lines.append(f"{self.CAPTURE_IMAGE_CMD} ; Capture image at this position")
```

Finaliza o programa

```
gcode_lines.append("\nM2 ; End program")
```

```
return "\n".join(gcode_lines)
```

def read_gcode_file(self, filename):

"""

Lê um arquivo G-CODE e cria uma sequência de posições de inspeção

Args:

filename: Caminho do arquivo G-CODE

Returns:

Um dicionário contendo:

- 'sequence_name': Nome da sequência extraída do arquivo
- 'positions': Lista de dicionários com informações das posições

"""

if not os.path.exists(filename):

logger.error(f'Arquivo G-CODE não encontrado: {filename}')

return None

try:

with open(filename, 'r') as f:

gcode_content = f.read()

return self.parse_gcode(gcode_content)

except Exception as e:

logger.error(f'Erro ao ler arquivo G-CODE: {str(e)}')

return None

def parse_gcode(self, gcode_content):

"""

Analisa o conteúdo G-CODE e extrai sequência e posições

Args:

gcode_content: String contendo o G-CODE a ser analisado

Returns:

Dicionário com dados da sequência e posições

"""

lines = gcode_content.split('\n')

sequence_name = "Sequência Importada"

positions = []

current_position = None

Extrai nome da sequência

for line in lines:

if self.SEQUENCE_NAME_CMD in line:

sequence_name = line.split(self.SEQUENCE_NAME_CMD, 1)[1].strip()

break

Extrai as posições

for line in lines:

Nova posição

if self.POSITION_NAME_CMD in line:

Finaliza posição anterior se existir

```

if current_position:
    positions.append(current_position)

# Inicia nova posição
position_name = line.split(self.POSITION_NAME_CMD, 1)[1].strip()
current_position = {
    'name': position_name,
    'x': 0.0,
    'y': 0.0,
    'camera_params': {}
}

# Parâmetros da câmera
elif self.CAMERA_PARAMS_CMD in line and current_position:
    params_str = line.split(self.CAMERA_PARAMS_CMD, 1)[1].strip()
    params_pairs = params_str.split(',')

    for pair in params_pairs:
        if ':' in pair:
            key, value = pair.split(':', 1)
            # Tenta converter para número se possível
            try:
                if '.' in value:
                    value = float(value)
                else:
                    value = int(value)
            except ValueError:
                pass # Mantém como string

            current_position['camera_params'][key] = value

# Movimento G0/G1
elif (line.strip().startswith('G0') or line.strip().startswith('G1')) and current_position:
    # Extraí coordenadas X e Y
    x_match = re.search(r'X([-\\d.]+)', line)
    y_match = re.search(r'Y([-\\d.]+)', line)

    if x_match:
        current_position['x'] = float(x_match.group(1))
    if y_match:
        current_position['y'] = float(y_match.group(1))

# Adiciona a última posição se existir
if current_position:
    positions.append(current_position)

return {
    'sequence_name': sequence_name,
    'positions': positions
}

```

```

def save_gcode_to_file(self, sequence, filename):
    """
    Gera G-CODE a partir de uma sequência e salva em um arquivo

    Args:
        sequence: Objeto InspectionSequence contendo as posições
        filename: Caminho do arquivo onde o G-CODE será salvo

    Returns:
        Boolean indicando sucesso da operação
    """
    try:
        gcode_content = self.generate_gcode_from_sequence(sequence)

        with open(filename, 'w') as f:
            f.write(gcode_content)

        return True

    except Exception as e:
        logger.error(f"Erro ao salvar arquivo G-CODE: {str(e)}")
        return False

```

File: position_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
```

```

class InspectionPosition:
    """Representa uma posição de inspeção."""

    def __init__(self, name, x, y, camera_params=None):
        self.name = name
        self.x = x
        self.y = y
        self.camera_params = camera_params or {}
        self.creation_time = time.time()

    def to_dict(self):
        """Converte para dicionário para serialização."""
        return {
            'name': self.name,
            'x': self.x,
            'y': self.y,
            'camera_params': self.camera_params,
            'creation_time': self.creation_time
        }

    @classmethod
    def from_dict(cls, data):

```

```

"""Cria a partir de um dicionário."""
position = cls(data['name'], data['x'], data['y'], data['camera_params'])
position.creation_time = data.get('creation_time', time.time())
return position

```

class InspectionSequence:

```

"""Representa uma sequência de posições de inspeção."""

```

```

def __init__(self, name):
    self.name = name
    self.positions = []
    self.creation_time = time.time()
    self.last_run_time = None

```

```

def add_position(self, position):
    """Adiciona uma posição à sequência."""
    self.positions.append(position)

```

```

def remove_position(self, index):
    """Remove uma posição pelo índice."""
    if 0 <= index < len(self.positions):
        del self.positions[index]
        return True
    return False

```

```

def move_position(self, old_index, new_index):
    """Move uma posição na sequência."""
    if 0 <= old_index < len(self.positions) and 0 <= new_index < len(self.positions):
        position = self.positions.pop(old_index)
        self.positions.insert(new_index, position)
        return True
    return False

```

```

def to_dict(self):
    """Converte para dicionário para serialização."""
    return {
        'name': self.name,
        'positions': [p.to_dict() for p in self.positions],
        'creation_time': self.creation_time,
        'last_run_time': self.last_run_time
    }

```

@classmethod

```

def from_dict(cls, data, positions_dict=None):
    """Cria a partir de um dicionário."""
    sequence = cls(data['name'])
    sequence.creation_time = data.get('creation_time', time.time())
    sequence.last_run_time = data.get('last_run_time')

```

```

# Reconstruir posições

```

```

for pos_data in data['positions']:
    if positions_dict and pos_data['name'] in positions_dict:
        # Usa posição existente
        sequence.positions.append(positions_dict[pos_data['name']])
    else:
        # Cria nova posição
        position = InspectionPosition.from_dict(pos_data)
        sequence.positions.append(position)

return sequence

```

```

class InspectionPositionManager:
    """Gerencia posições e sequências de inspeção."""

    def __init__(self):
        self.positions = {} # name -> position
        self.sequences = {} # name -> sequence

    def add_position(self, position):
        """Adiciona uma posição."""
        self.positions[position.name] = position

    def remove_position(self, position_name):
        """Remove uma posição pelo nome."""
        if position_name in self.positions:
            del self.positions[position_name]

            # Atualiza sequências que usam esta posição
            for sequence in self.sequences.values():
                sequence.positions = [p for p in sequence.positions if p.name != position_name]

        return True
    return False

    def get_position(self, position_name):
        """Obtém uma posição pelo nome."""
        return self.positions.get(position_name)

    def add_sequence(self, sequence):
        """Adiciona uma sequência."""
        self.sequences[sequence.name] = sequence

    def remove_sequence(self, sequence_name):
        """Remove uma sequência pelo nome."""
        if sequence_name in self.sequences:
            del self.sequences[sequence_name]
            return True
        return False

    def get_sequence(self, sequence_name):

```

```
"""Obtém uma sequência pelo nome."""  
return self.sequences.get(sequence_name)
```

```
def save_to_file(self, filename):  
    """Salva todas as posições e sequências em um arquivo JSON."""  
    data = {  
        'positions': {name: pos.to_dict() for name, pos in self.positions.items()},  
        'sequences': {name: seq.to_dict() for name, seq in self.sequences.items()}  
    }  
  
    try:  
        with open(filename, 'w') as f:  
            import json  
            json.dump(data, f, indent=2)  
        return True  
    except Exception as e:  
        print(f"Erro ao salvar dados: {e}")  
        return False
```

```
def load_from_file(self, filename):  
    """Carrega posições e sequências de um arquivo JSON."""  
    try:  
        with open(filename, 'r') as f:  
            import json  
            data = json.load(f)  
  
        # Limpa dados existentes  
        self.positions.clear()  
        self.sequences.clear()  
  
        # Carrega posições  
        for name, pos_data in data.get('positions', {}).items():  
            position = InspectionPosition.from_dict(pos_data)  
            self.positions[name] = position  
  
        # Carrega sequências  
        for name, seq_data in data.get('sequences', {}).items():  
            sequence = InspectionSequence.from_dict(seq_data, self.positions)  
            self.sequences[name] = sequence  
  
        return True  
  
    except Exception as e:  
        print(f"Erro ao carregar dados: {e}")  
        return False
```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
# Exporta as classes principais para facilitar importação
from .aoi_controller import CNCAOIController
from .cnc_controller import GRBLCNCController
from .camera_controller import CameraController
from .position_manager import InspectionPosition, InspectionSequence

__version__ = '0.1.0'
```

File: config_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils